

UNIVERSIDAD EUROPEA DEL ATLÁNTICO
ESCUELA POLITÉCNICA SUPERIOR
GRADO EN INGENIERÍA INFORMÁTICA



TRABAJO FINAL DE GRADO

Desarrollo de una plataforma web integral de gestión de citas para la peluquería
Corte Perfecto con asistencia inteligente basada en modelos de lenguaje (LLM) de
ejecución local

Trabajo fin de grado realizado por
Adrián García Arranz

bajo la dirección de
David García Obeso

SANTANDER – Junio, 2026

**EUROPEAN UNIVERSITY OF THE ATLANTIC
HIGHER POLYTECHNIC SCHOOL
DEGREE IN COMPUTER ENGINEERING**



DISSERTATION

Development of an Integrated Web Platform for Appointment Management for the
Corte Perfecto Hair Salon with Intelligent Assistance Based on Locally Executed
Language Models (LLM)

Dissertation carried out by
Adrián García Arranz

supervised by
David García Obeso

SANTANDER – June 2026

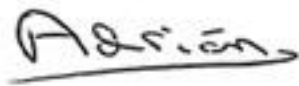
DECLARACIÓN DE ORIGINALIDAD DEL TFG

D. Adrián García Arranz, con DNI 72203262Y, declaro que el presente Trabajo Fin de Grado es original y todas las fuentes de información han sido debidamente citadas, no habiendo incurrido en plagio, según lo estipulado en la normativa general para los Trabajos de Fin de Grado de la Universidad Europea del Atlántico.

Y para que así conste, lo suscribo y firmo.

En Santander, a 12 de junio de 2026

Firma del alumno

A handwritten signature in black ink, appearing to read 'Adrián', with a horizontal line underneath it.

Resumen

Este Trabajo de Fin de Grado presenta el diseño, desarrollo y validación de una plataforma web integral para la peluquería Corte Perfecto, ubicada en Santander. La solución digitaliza la consulta de servicios y la gestión de citas mediante una aplicación full-stack compuesta por un frontend React/Vite, un backend Node.js/Express y una base de datos MongoDB. Como aportación principal, el sistema incorpora un asistente conversacional conectado a LM Studio, capaz de atender al cliente en lenguaje natural y apoyar el proceso de reserva sin enviar datos personales a servicios externos. El trabajo documenta el escenario de partida, el estado del arte, los requisitos funcionales y no funcionales, el análisis, el diseño técnico, la implementación y la verificación final. La arquitectura mantiene una separación clara entre interfaz, reglas de negocio, persistencia e integración con inteligencia artificial local, de modo que el modelo de lenguaje aporta naturalidad conversacional mientras el backend conserva la autoridad sobre validaciones, disponibilidad, precios, solapes y persistencia. Los resultados muestran una solución funcional, trazable y mantenible, adecuada para automatizar reservas en un pequeño negocio local, mejorar la disponibilidad del servicio para el cliente y reducir tareas administrativas repetitivas del profesional.

Palabras clave: React, Node.js, MongoDB, inteligencia artificial local, gestión de citas.

Abstract

This Bachelor Degree Final Project presents the design, development and validation of an integrated web platform for Corte Perfecto, a hair salon located in Santander. The solution digitalizes service consultation and appointment management through a full-stack application composed of a React/Vite frontend, a Node.js/Express backend and a MongoDB database. Its main contribution is the integration of a conversational assistant connected to LM Studio, which supports customers in natural language and helps complete bookings without sending personal data to external artificial intelligence services. The dissertation documents the initial scenario, state of the art, functional and non-functional requirements, analysis, technical design, implementation and final verification. The architecture clearly separates user interface, business rules, persistence and local artificial intelligence integration, so the language model provides conversational flexibility while the backend remains responsible for validation, availability, prices, overlaps and data persistence. The results show a functional, traceable and maintainable solution suitable for automating bookings in a small local business, improving service availability for customers and reducing repetitive administrative tasks for the professional.

Keywords: React, Node.js, MongoDB, local artificial intelligence, appointment management.

Índice

CAPÍTULO 1. INTRODUCCIÓN, ESTADO DEL ARTE, OBJETIVOS Y METODOLOGÍA	1
1.1 Introducción, Escenario y Marco Teórico	1
1.1.1 Contextualización del escenario	1
1.1.2 La oportunidad de la Inteligencia Artificial conversacional	2
1.1.3 Actores del sistema	3
1.1.4 Marco teórico	4
1.2 Estado del Arte	5
1.2.1 Gestión manual clásica: papel, agenda y llamadas telefónicas	5
1.2.2 WhatsApp: solución aparente y trampa operativa	5
1.2.3 Plataformas SaaS especializadas en reservas	6
1.2.4 Modelos de lenguaje de código abierto y su viabilidad local	7
1.2.5 Chatbots y procesamiento de lenguaje natural en servicios locales	7
1.2.6 Diferenciación de este trabajo respecto al estado del arte	7
1.3 Objetivos	8
1.3.1 Hipótesis de partida	8
1.3.2 Objetivo general	8
1.3.3 Objetivos específicos y correspondencia con los capítulos	9
1.4 Metodología	10
1.4.1 Enfoque metodológico general	10
1.4.2 Ciclo de vida por funcionalidad	11
1.4.3 Fases del proyecto	11
1.4.4 Tecnologías utilizadas y justificación	12
1.4.5 Consideraciones sobre privacidad y cumplimiento del RGPD	13
1.4.6 Trazabilidad metodológica y evidencia de construcción	14
1.5 Alcance y Limitaciones	14
1.5.1 Alcance funcional del sistema	14

1.5.2 Limitaciones actuales y líneas de evolución	15
Resumen del Capítulo	16
CAPÍTULO 2. DISCIPLINA DE REQUISITOS	17
2.1 Modelo del dominio	17
2.1.1 Diagrama de clases del dominio	17
2.1.2 Diagrama de objetos	19
2.1.3 Diagrama de estados	20
2.1.4 Requisitos suplementarios	21
2.2 Diagrama de contexto.....	23
2.3 Actores del sistema.....	23
2.4 Casos de uso	24
2.4.1 Diagrama de casos de uso	24
2.4.2 Priorización de casos de uso	27
2.4.3 Detalle de casos de uso	27
2.4.4 Diagramas de comportamiento de casos críticos	28
2.4.5 Matriz de trazabilidad RS-UC	33
2.4.6 Prototipos de los casos de uso.....	34
2.4.7 Trazabilidad RUP y seguimiento de casos de uso	34
2.4.8 Criterios de aceptación verificables	34
Resumen del capítulo	35
CAPÍTULO 3. ANÁLISIS Y DISEÑO	36
3.1 Disciplina de análisis.....	36
3.1.1 Análisis de la arquitectura.....	36
3.1.2 Análisis de casos de uso.....	40
Análisis UC-02/03: consultar servicios, precios y detalle de opción.....	41
Análisis UC-05: reservar cita por chatbot.....	42
Análisis UC-09: modificar reserva activa por chat.....	43
Análisis UC-10: iniciar sesión administrador	45
Análisis UC-12/13/14/15/16: gestión administrativa de citas	46
3.1.3 Análisis de clases	47

3.1.4 Análisis de paquetes.....	47
3.2 Disciplina de diseño	48
3.2.1 Diseño de la arquitectura	48
3.2.2 Diseño del modelo de datos	53
3.2.3 Diseño de clases y módulos	54
Patrones y principios aplicados:.....	56
3.2.4 Diseño de paquetes	57
3.2.5 Diseño de interfaces de usuario	58
3.2.6 Diseño de integración con sistemas locales	58
3.2.7 Estructura del system prompt y contingencia de IA local	61
3.3 Trazabilidad y planificación técnica	62
Medidas de calidad de diseño.....	62
3.4 Auditoría diseño-implementación	63
3.5 Plan de pruebas automatizadas.....	63
3.6 Dashboard RUP del proyecto.....	64
Resumen del capítulo	64
Capítulo 4. Implementación y mapa de la solución.....	65
4.1 Estado técnico de la solución implementada.....	65
4.2 Organización final del código	65
4.3 Mapa navegable de la solución	66
4.4 Web pública: escaparate y entrada al chatbot	69
4.5 Chatbot de reserva con LM Studio.....	70
4.6 Acceso y panel de administración.....	72
4.7 Casos de uso representativos resueltos en interfaz.....	76
4.8 Persistencia y sincronización de datos	77
4.9 Verificación de la implementación	788
4.10 Recorrido end-to-end de una reserva	78
4.11 Revisión MVC y principios de diseño	79
4.12 Cierre del capítulo	79
Capítulo 5. Conclusiones y líneas futuras.....	811

DESARROLLO DE UNA PLATAFORMA WEB INTEGRAL DE GESTIÓN DE CITAS PARA LA PELUQUERÍA CORTE
PERFECTO CON ASISTENCIA INTELIGENTE BASADA EN MODELOS DE LENGUAJE (LLM) DE EJECUCIÓN
LOCAL

5.1 Cumplimiento del objetivo general	811
5.2 Cumplimiento de objetivos específicos.....	811
5.3 Cumplimiento de objetivos transversales.....	822
5.4 Evaluación de eficiencia e integridad.....	822
5.5 Discusión de resultados	833
5.6 Limitaciones detectadas	844
5.7 Recomendaciones.....	855
5.8 Futuras líneas de actuación	866
5.9 Valoración personal del proceso	877
5.10 Conclusión final	877
Referencias.....	888

Índice de figuras

Figura 2.1	Diagrama de clases del dominio de Corte Perfecto	18
Figura 2.2	Diagrama de objetos de una reserva desde conversación.....	20
Figura 2.3	Estados principales de una cita.....	21
Figura 2.4	Diagrama de contexto del sistema.....	23
Figura 2.5	Casos de uso del cliente.....	25
Figura 2.6	Casos de uso del administrador	26
Figura 2.7	Actividad principal de reserva por chatbot.....	30
Figura 2.8	Secuencia técnica de confirmación de cita por chat	31
Figura 2.9	Actividad de gestión administrativa de citas	32
Figura 3.1	Arquitectura lógica por capas de Corte Perfecto	39
Figura 3.2	Secuencia de consulta de servicios, precios y detalle de opción	42
Figura 3.3	Secuencia de reserva de cita mediante chatbot.....	43
Figura 3.4	Secuencia de modificación de una reserva activa por chat.....	44
Figura 3.5	Secuencia de inicio de sesión del administrador	45
Figura 3.6	Secuencia de gestión administrativa de citas.....	46
Figura 3.7	Clases de análisis clasificadas por MVC.....	47
Figura 3.8	Paquetes de análisis y dependencias.....	48
Figura 3.9	Arquitectura técnica de componentes y protocolos	50
Figura 3.10	Diseño de despliegue local	52
Figura 3.11	Modelo de datos documental en MongoDB	53
Figura 3.12	Diseño simplificado de clases y módulos del backend de reserva	55
Figura 3.13	Integración del cliente con chatbot, backend, LM Studio y MongoDB	60
Figura 4.1	Diagrama bidireccional de navegación por casos de uso	67
Figura 4.2	Mapa navegable de la solución implementada	68
Figura 4.3	Pantalla de inicio de Corte Perfecto	69
Figura 4.4	Widget de chat abierto sobre la web pública.....	70
Figura 4.5	Flujo técnico de reserva por chatbot.....	71
Figura 4.6	Pantalla de inicio de sesión del administrador.....	73
Figura 4.7	Dashboard administrativo.....	74
Figura 4.8	Gestión administrativa de citas.....	75
Figura 4.9	Formulario de creación manual de cita.....	76

Índice de tablas

Tabla 1	1.2.3 Plataformas SaaS especializadas en reservas	6
Tabla 2	1.3.3 Objetivos específicos y correspondencia con los capítulos	9
Tabla 3	1.4.3 Fases del proyecto	12
Tabla 4	1.4.4 Tecnologías utilizadas y justificación	12
Tabla 5	2.1.4 Requisitos suplementarios.....	22
Tabla 6	2.3 Actores del sistema	24
Tabla 7	2.4.2 Priorización de casos de uso	27
Tabla 8	2.4.5 Matriz de trazabilidad RS-UC.....	33
Tabla 9	2.4.8 Criterios de aceptación verificables	34
Tabla 10	Comparación de estilos arquitectónicos.....	37
Tabla 11	Capas de la arquitectura	38
Tabla 12	3.1.2 Análisis de casos de uso.....	40
Tabla 13	Componentes de la arquitectura	50
Tabla 14	Módulos y responsabilidades	55
Tabla 15	3.2.6 Diseño de integración con sistemas locales	59
Tabla 16	3.2.7 Estructura del system prompt y contingencia de IA local.....	61
Tabla 17	3.5 Plan de pruebas automatizadas	63
Tabla 18	4.2 Organización final del código	66
Tabla 19	4.3 Mapa navegable de la solución	68
Tabla 20	4.5 Chatbot de reserva con LM Studio.....	71
Tabla 21	Casos de uso representativos resueltos en interfaz	76
Tabla 22	4.8 Persistencia y sincronización de datos	77
Tabla 23	4.9 Verificación de la implementación	78
Tabla 24	4.10 Recorrido end-to-end de una reserva	799
Tabla 25	5.2 Cumplimiento de objetivos específicos	811
Tabla 26	5.4 Evaluación de eficiencia e integridad	833
Tabla 27	5.5 Discusión de resultados.....	844
Tabla 28	5.8 Futuras líneas de actuación	866

CAPÍTULO 1. INTRODUCCIÓN, ESTADO DEL ARTE, OBJETIVOS Y METODOLOGÍA

1.1 INTRODUCCIÓN, ESCENARIO Y MARCO TEÓRICO

La narrativa de este capítulo se construye siguiendo una aproximación por etapas que facilita la comprensión progresiva del proyecto: (1) contextualización del escenario real de la peluquería y su problemática operativa; (2) exploración de las soluciones existentes en el mercado y sus limitaciones; (3) justificación de por qué ninguna de esas soluciones se adapta de forma óptima al caso concreto; y (4) presentación de la propuesta específica desarrollada en este trabajo, que sirve como puente natural hacia la hipótesis y los objetivos.

1.1.1 Contextualización del escenario

El sector de la peluquería y la estética personal en España constituye un tejido empresarial formado mayoritariamente por pequeños negocios de carácter local o familiar. Según datos del Instituto Nacional de Estadística, existen en el país más de 60.000 establecimientos de peluquería y cuidado personal, de los cuales una parte muy significativa opera con plantillas de entre uno y cinco empleados. Esta realidad implica que la gestión administrativa del negocio —en particular la gestión de citas con clientes— recaea directamente sobre el propio profesional, sin que exista habitualmente un equipo dedicado exclusivamente a esa tarea.

En este contexto se sitúa el negocio objeto de este proyecto: "Corte Perfecto", una peluquería de barrio ubicada en la ciudad de Santander (Cantabria). El establecimiento ofrece tres servicios principales: corte de cabello (20 €, incluye lavado), tinte o coloración capilar (40 €) y peinado o recogido (15 €). Funciona en horario de lunes a viernes, de 10:00 a 20:00, con cierre total los fines de semana. Al igual que la mayoría de negocios de su categoría,

gestiona su agenda mediante métodos tradicionales: anotaciones en papel, mensajes de WhatsApp y llamadas telefónicas.

El perfil de la clientela es heterogéneo: clientes jóvenes o de mediana edad que prefieren resolver por chat y esperan respuestas rápidas; clientela tradicional que sigue llamando por teléfono; y clientes ocasionales que preguntan precios o disponibilidad antes de decidir. Esta mezcla de hábitos obliga a cubrir varios canales simultáneamente, lo que incrementa la carga cognitiva del peluquero y genera interrupciones constantes durante el trabajo.

La problemática concreta identificada es la siguiente: los clientes no disponen de un canal digital disponible las veinticuatro horas del día para consultar disponibilidad y formalizar una reserva, y el negocio carece de un sistema automatizado que persista esas reservas en una base de datos estructurada. La estimación de partida, documentada en la propuesta inicial del proyecto (D0), sitúa la pérdida de tiempo productivo derivada de la gestión manual de agenda y la atención de consultas repetitivas en torno al 30 % en periodos de alta demanda. Esta situación puede resumirse en una frase recurrente en el diagnóstico del negocio: "se pierden reservas no porque no haya hueco, sino porque no hay manos para contestar".

1.1.2 La oportunidad de la Inteligencia Artificial conversacional

La proliferación de los Modelos de Lenguaje de Gran Escala (Large Language Models, LLM) ha abierto una nueva posibilidad tecnológica: dotar a cualquier aplicación web de una interfaz conversacional en lenguaje natural. En lugar de obligar al usuario a rellenar formularios o navegar menús, estos modelos permiten que el cliente simplemente

escriba lo que necesita y que el sistema entienda la intención, extraiga los datos relevantes y actúe en consecuencia.

Sin embargo, existe una barrera relevante para su adopción en pequeños negocios: el coste y la privacidad. Los servicios comerciales de IA —como GPT-4 (OpenAI) o Claude (Anthropic)— implican costes por uso que se acumulan con el volumen de conversaciones, y suponen enviar datos de los clientes a servidores externos fuera del control del negocio. Este aspecto es especialmente sensible en el marco del Reglamento General de Protección de Datos (RGPD) de la Unión Europea.

La aparición de modelos de código abierto de alta calidad —en particular la familia Llama 3.1 de Meta AI (2024)— y de herramientas de inferencia local como LM Studio ofrece una alternativa viable: ejecutar un LLM de forma completamente local, en el propio hardware del negocio, sin enviar ningún dato a servicios externos. Esta aproximación elimina el coste por petición y mantiene la privacidad de los datos de los clientes bajo control directo del negocio. El hardware disponible —un equipo con GPU NVIDIA RTX 3060 de 6 GB de VRAM, documentado en D0— hace técnicamente viable la ejecución del modelo Llama 3.1 8B en versión cuantizada a 4 bits (formato GGUF).

1.1.3 Actores del sistema

La solución se diseña en torno a dos actores principales cuyas necesidades definen los requisitos funcionales del sistema:

- **Cliente final:** persona que desea consultar información sobre el negocio (horarios, precios, servicios) o reservar una cita. Accede a la plataforma desde el navegador web, preferentemente desde el teléfono móvil, e interactúa mediante el widget de chat en lenguaje natural.

- Administrador del negocio (peluquero): profesional que necesita gestionar la agenda.

Accede al panel de administración de la plataforma para visualizar, modificar o cancelar las reservas registradas en la base de datos.

1.1.4 Marco teórico

Para situar el proyecto en su contexto académico y tecnológico, se describen los conceptos fundamentales sobre los que se apoya el desarrollo:

Modelos de lenguaje de gran escala (LLM). Un LLM es un modelo de aprendizaje profundo entrenado sobre corpus masivos de texto que adquiere la capacidad de generar texto coherente y seguir instrucciones en lenguaje natural. Su arquitectura está basada en el mecanismo de transformers (Vaswani et al., 2017). En este proyecto se utiliza Llama 3.1 8B Instruct, distribuido bajo licencia abierta por Meta AI y optimizado para mantener conversaciones estructuradas.

Inferencia local con LM Studio. LM Studio es una aplicación de escritorio que permite cargar y ejecutar modelos LLM localmente, exponiendo una API compatible con el estándar OpenAI. El modelo se ejecuta en la GPU local mediante cuantización a 4 bits (formato GGUF), lo que reduce su huella de memoria de aproximadamente 16 GB (float16) a 5-6 GB, haciéndolo operable en la RTX 3060 (LM Studio, s. f.).

Arquitectura cliente-servidor. La plataforma sigue el patrón de separación entre capa de presentación (frontend React ejecutado con Vite durante el desarrollo), lógica de negocio (backend Node.js/Express organizado siguiendo MVC) y capa de datos (MongoDB gestionada mediante Mongoose). Esta separación facilita el mantenimiento, la escalabilidad y la reutilización de componentes (Express.js, s. f.; MongoDB, Inc., s. f.; Mongoose, s. f.; OpenJS Foundation, s. f.; React, s. f.; Vite, s. f.).

IA guiada por reglas de negocio. Un principio de diseño fundamental es que la IA conversa, pero el backend valida y decide. El backend incorpora un flujo determinista para la reserva de citas y utiliza el LLM como asistente conversacional, no como autoridad final del sistema. Antes de registrar una cita, la aplicación verifica que el nombre sea válido, que la fecha no caiga en fin de semana, que la hora pertenezca al horario laboral y no haya pasado, que el servicio exista en el catálogo numerado y que no se generen reservas duplicadas. Este enfoque evita el error habitual de tratar la IA como una "caja mágica" sin supervisión.

1.2 ESTADO DEL ARTE

1.2.1 Gestión manual clásica: papel, agenda y llamadas telefónicas

La gestión tradicional mediante agenda física, llamadas directas y anotaciones en papel ha sido el método predominante en el sector durante décadas. Sus ventajas iniciales son evidentes: no requiere inversión tecnológica ni formación compleja. Sin embargo, cuando aumenta la demanda o se diversifican los canales de comunicación, emergen limitaciones estructurales: ausencia de sincronización automática entre consultas y citas confirmadas; dependencia de quién anota y en qué soporte; imposibilidad de atender solicitudes fuera del horario de trabajo; y falta de trazabilidad para revisar cancelaciones.

1.2.2 WhatsApp: solución aparente y trampa operativa

WhatsApp se percibe habitualmente como alternativa gratuita y cómoda porque casi todos los clientes lo usan. Sin embargo, en operación real actúa como una trampa para el pequeño negocio: su coste no es monetario sino temporal. El profesional debe responder mensajes fuera de horario para no perder conversaciones; los hilos de múltiples clientes se mezclan y son difíciles de gestionar; y no existe ningún mecanismo de persistencia

estructurada. Cuando el volumen de consultas supera cierto umbral, WhatsApp se convierte en una fuente de interrupciones más que en una herramienta de productividad.

1.2.3 Plataformas SaaS especializadas en reservas

Existe un ecosistema consolidado de plataformas Software como Servicio (SaaS) orientadas a la gestión de citas en negocios de peluquería y estética. La siguiente tabla comparativa analiza las más representativas:

Tabla 1

1.2.3 Plataformas SaaS especializadas en reservas

Solución	Tipo	Ventajas	Limitaciones
Fresha	SaaS peluquería	Agenda, clientes y pagos integrados	Costes en plan de pago; sin IA conversacional; datos en terceros
Booksy	App reservas	Amplia base de usuarios; valoraciones	Coste mensual; datos en servidores externos
SimplyBook.me	SaaS genérico	Alta configurabilidad; múltiples servicios	Complejidad de configuración; sin LLM integrado
WhatsApp Business	Mensajería empresarial	Familiar para el cliente; sin instalación	Manual al 100 %; sin persistencia automática
Calendly	Reuniones B2B	Fácil integración; tier gratuito	No orientado a peluquerías; sin lenguaje natural
Este TFG	Web ad hoc + IA local	Privacidad, sin costes recurrentes, IA conversacional integrada	Requiere desarrollo e infraestructura inicial

Ninguna de las soluciones comerciales analizadas combina simultáneamente las tres características que este proyecto persigue: (1) interfaz conversacional en lenguaje natural, (2)

ejecución local del modelo de IA sin dependencia de terceros, y (3) persistencia automática y estructurada de las citas en una base de datos propia del negocio.

1.2.4 Modelos de lenguaje de código abierto y su viabilidad local

La publicación de Llama 2 (Touvron et al., 2023) y posteriormente Llama 3.1 (Meta AI, 2024) supuso un punto de inflexión en la accesibilidad de los LLMs. La cuantización a 4 bits (formato GGUF, implementado a través de llama.cpp y LM Studio) reduce el uso de memoria de un modelo de 8.000 millones de parámetros de aproximadamente 16 GB en float16 a 5-6 GB, haciéndolo operable en hardware de consumo estándar. Este proyecto verifica experimentalmente esa viabilidad en una RTX 3060 con 6 GB de VRAM, obteniendo tiempos de inferencia de entre 2 y 8 segundos por respuesta, aceptables para un chatbot de atención al cliente.

1.2.5 Chatbots y procesamiento de lenguaje natural en servicios locales

Følstad y Brandtzæg (2017) identifican la gestión de citas como uno de los principales casos de uso de los chatbots en atención al cliente, señalando que la principal barrera para la adopción es la rigidez de las interfaces basadas en menús de selección, que limitan la expresividad del usuario. Los LLMs modernos superan esta limitación: el usuario puede escribir "quiero un corte para el miércoles por la mañana" y el sistema extrae inteligentemente el servicio, la fecha y la preferencia horaria. Esta capacidad —conocida como slot filling en el ámbito del procesamiento del lenguaje natural— constituye el núcleo funcional del chatbot de este proyecto.

1.2.6 Diferenciación de este trabajo respecto al estado del arte

Este proyecto se diferencia de las soluciones existentes en cuatro aspectos clave:

- Privacidad por diseño: al ejecutar el LLM localmente, ningún dato del cliente — nombre, fecha de cita, historial de conversación— abandona el entorno controlado por el negocio, facilitando el cumplimiento del RGPD.
- Coste cero de operación del componente de IA: a diferencia de las APIs comerciales (OpenAI, Anthropic, Google), el coste de inferencia es nulo tras la inversión inicial en hardware.
- Integración directa con base de datos: el chatbot crea automáticamente el registro de la cita al completar la conversación, eliminando la intervención manual del profesional.
- Control mediante ingeniería de prompts y validación backend: todas las decisiones críticas (validación de nombre, bloqueo de fechas en fin de semana, cálculo de precios combinados, deduplicación) se gestionan en el backend, no en el modelo de IA.

1.3 OBJETIVOS

1.3.1 Hipótesis de partida

El desarrollo de una plataforma web que integre un chatbot de inteligencia artificial conversacional ejecutado de forma local puede automatizar el proceso de gestión de citas de una peluquería pequeña, ofreciendo al cliente un canal disponible las veinticuatro horas del día, garantizando la privacidad de los datos de los usuarios y sin incurrir en costes recurrentes por el uso del componente de inteligencia artificial.

1.3.2 Objetivo general

Diseñar e implementar una plataforma web de gestión de reservas para la peluquería "Corte Perfecto" (Santander) que automatice la atención al cliente mediante el despliegue de un asistente conversacional basado en un modelo de lenguaje de gran escala ejecutado en

entorno local, persistiendo las reservas automáticamente en MongoDB y proporcionando al negocio una interfaz de administración para la gestión de la agenda.

1.3.3 Objetivos específicos y correspondencia con los capítulos

Los objetivos específicos están directamente alineados con la estructura de capítulos de este documento, siguiendo el mapa propuesto en la guía de la Escuela Politécnica Superior:

Tabla 2

1.3.3 Objetivos específicos y correspondencia con los capítulos

#	Objetivo específico	Capítulo asociado
OE1	Ejecutar la disciplina de requisitos: identificar los requisitos funcionales y no funcionales del sistema, modelar los casos de uso y definir el contrato de la API REST entre frontend y backend.	Capítulo 2
OE2	Ejecutar la disciplina de análisis y diseño: definir la arquitectura del sistema (React/Vite + Node.js/Express + MongoDB + LM Studio), el modelo de datos documental y el diseño del flujo de reserva y limpieza de respuestas del LLM.	Capítulo 3
OE3	Desarrollar un producto final funcional que satisfaga los requisitos identificados: frontend React/Vite, backend Node.js/Express con estructura MVC, base de datos MongoDB y chatbot con Llama 3.1 8B Instruct.	Capítulo 4
OE4	Evaluar la eficiencia del sistema: medir los tiempos de respuesta del LLM, verificar la integridad de las transacciones en la base de datos y validar el cumplimiento de todos los objetivos anteriores.	Capítulo 5

El Capítulo 5 recoge la verificación del cumplimiento de cada uno de estos objetivos, junto con las conclusiones del trabajo, las limitaciones encontradas y las futuras líneas de actuación.

De forma transversal a todos los capítulos, se establecen los siguientes objetivos complementarios:

- OET1 — Privacidad: garantizar que ningún dato personal del cliente sea enviado a servicios de IA externos durante la operación del sistema.
- OET2 — Robustez del chatbot: implementar un flujo conversacional con validaciones backend, catálogo numerado y limpieza de respuestas del LLM para garantizar que el usuario nunca reciba texto de razonamiento interno del modelo.
- OET3 — Usabilidad: lograr que un usuario sin conocimientos técnicos pueda completar una reserva en cinco o menos intercambios de mensajes con el chatbot.
- OET4 — Mantenibilidad: estructurar el código con separación clara de responsabilidades (frontend / backend / IA) para facilitar futuras extensiones.

1.4 METODOLOGÍA

1.4.1 Enfoque metodológico general

El desarrollo de este proyecto sigue una metodología de ciclo de vida iterativo e incremental, enmarcada en los principios del desarrollo ágil de software descritos por Pressman y Maxim (2020). Este enfoque fue seleccionado por dos razones fundamentales.

En primer lugar, la naturaleza del componente de inteligencia artificial introduce una incertidumbre inherente: el comportamiento exacto del LLM ante diferentes entradas no es completamente predecible desde el diseño inicial y requiere ciclos cortos de prueba y ajuste. Durante el desarrollo se identificaron situaciones no anticipadas en el análisis original, como la necesidad de gestionar fechas expresadas en lenguaje coloquial ("este miércoles", "pasado mañana"), la combinación de múltiples servicios en una única reserva, o los casos en los que el modelo omite el marcador de respuesta establecido en el system prompt.

En segundo lugar, los requisitos del negocio, aunque claros en términos generales, evolucionaron durante el desarrollo al descubrirse nuevos casos límite. La metodología iterativa permite incorporar estas modificaciones sin alterar la estructura global del proyecto.

1.4.2 Ciclo de vida por funcionalidad

Dentro de cada fase, se aplicó el siguiente ciclo de vida para cada funcionalidad implementada, consistente con el modelo iterativo descrito por Pressman y Maxim (2020):

- Definición: especificación precisa del comportamiento esperado.
- Implementación: codificación de la funcionalidad en backend, frontend o system prompt.
- Prueba manual: verificación del comportamiento con casos reales de uso.
- Ajuste: corrección de desviaciones respecto al comportamiento esperado.
- Integración: incorporación al sistema completo y verificación de ausencia de regresiones.

Este ciclo se aplicó especialmente al componente de chatbot. Cada modificación del system prompt o del pipeline de limpieza de respuestas requirió múltiples sesiones de prueba conversacional antes de ser considerada estable. La revisión humana continua (human-in-the-loop) fue esencial para detectar errores de comprensión que los tests automatizados no habrían identificado.

1.4.3 Fases del proyecto

El proyecto se organiza en cuatro fases principales, directamente correspondientes con la estructura de capítulos de este documento:

Tabla 3

1.4.3 Fases del proyecto

Fase	Contenido principal	Capítulo
Fase 1 — Requisitos	Análisis del negocio. Identificación de requisitos funcionales (RF) y no funcionales (RNF). Casos de uso. Definición del contrato de la API REST.	Capítulo 2
Fase 2 — Análisis y Diseño	Arquitectura de tres capas (React/Vite + Node.js/Express + MongoDB + LM Studio). Modelo documental. Diseño del widget de chat. Diseño del system prompt y del flujo determinista de reserva con limpieza de respuestas del LLM.	Capítulo 3
Fase 3 — Implementación	Desarrollo del frontend React/Vite. Backend Node.js/Express con endpoints de chat, citas, servicios y autenticación. Integración con LM Studio. Flujo de reserva validado por backend, catálogo numerado, filtro anti-metadatos y persistencia MongoDB. Pruebas.	Capítulo 4
Fase 4 — Evaluación y Conclusiones	Evaluación del cumplimiento de los objetivos. Medición de tiempos de respuesta del LLM. Validación de la integridad de la base de datos. Limitaciones y futuras líneas de trabajo.	Capítulo 5

1.4.4 Tecnologías utilizadas y justificación

Tabla 4

1.4.4 Tecnologías utilizadas y justificación

Tecnología	Rol en el proyecto	Justificación de la elección
React 19 + Vite 7	Frontend / interfaz de usuario	Componentes reutilizables; experiencia responsive; desarrollo rápido mediante Vite; estado reactivo para el chat y el panel de administración
Node.js + Express 5	Backend / API REST	Ecosistema JavaScript unificado con el frontend; API REST sencilla;

Tecnología	Rol en el proyecto	Justificación de la elección
MongoDB + Mongoose	Base de datos documental y ODM	middleware para seguridad, CORS, autenticación y validación Persistencia flexible para citas, servicios y administradores; esquemas Mongoose con validación; integración directa con el backend Node.js
LM Studio	Servidor de inferencia local	API compatible con OpenAI; interfaz gráfica para gestionar modelos; coste de operación cero
Llama 3.1 8B Instruct (Q4_K_M)	Modelo de lenguaje conversacional	Licencia comercial abierta; alta calidad en seguimiento de instrucciones; operable con 6 GB VRAM
Axios	Cliente HTTP de frontend y backend	Cliente HTTP utilizado para comunicarse con la API REST y con el servidor local de LM Studio
Git / GitHub	Control de versiones	Trazabilidad de cambios; estándar de la industria

1.4.5 Consideraciones sobre privacidad y cumplimiento del RGPD

Dado que el sistema almacena nombres de clientes y fechas de citas, el diseño incorpora desde el inicio los principios del Reglamento General de Protección de Datos (Reglamento [UE] 2016/679, 2016):

- Minimización de datos: el sistema solicita únicamente los datos estrictamente necesarios para la reserva: nombre, servicio, fecha y hora.
- Limitación de finalidad: los datos se usan exclusivamente para la gestión de citas y no se comparten con terceros.
- Privacidad por diseño: al procesar el lenguaje natural localmente con Llama 3.1, los datos de la conversación no abandonan el entorno del negocio en ningún momento.

- Derecho al olvido: la arquitectura permite al administrador eliminar registros de citas directamente desde el panel de administración, en cumplimiento del artículo 17 del RGPD.

1.4.6 Trazabilidad metodológica y evidencia de construcción

Además del desarrollo funcional, el proyecto incorpora una estructura de seguimiento inspirada en pySigHor, repositorio de referencia indicado por la dirección académica. Esta estructura permite mantener una trazabilidad explícita entre requisitos, análisis, diseño, implementación y pruebas, evitando que la memoria y el código evolucionen por caminos separados.

Este enfoque refuerza la metodología iterativa del proyecto: cada funcionalidad relevante no solo se implementa, sino que queda vinculada con su origen en requisitos y con una comprobación técnica cuando el riesgo lo justifica.

1.5 ALCANCE Y LIMITACIONES

1.5.1 Alcance funcional del sistema

El sistema desarrollado cubre el proceso completo desde el primer mensaje del cliente hasta la persistencia de la reserva en la base de datos:

- Consulta de horarios, precios y catálogo de servicios mediante lenguaje natural.
- Identificación automática de la intención de reserva.
- Recogida conversacional de los cuatro datos necesarios: nombre, servicio, fecha y hora.
- Cálculo automático de precios para combinaciones de servicios (hasta tres simultáneos).

- Modificación dinámica de una reserva activa (cambio de hora, adición de servicio).
- Bloqueo automático de fechas en fin de semana, con propuesta de alternativa laborable.
- Validación de nombre de cliente (rechazo de entradas inválidas como "ok" o "asdf").
- Deduplicación de reservas idénticas en la base de datos.
- Visualización del estado actual de la reserva mediante tarjeta visual en el chat.
- Panel de administración con autenticación para visualizar, crear, modificar, eliminar, ordenar y marcar como completadas las citas registradas.

1.5.2 Limitaciones actuales y líneas de evolución

Las siguientes limitaciones son razonables para el alcance de un TFG y dejan una base sólida para evolución futura:

- Base de datos MongoDB local: adecuada para desarrollo, demostración y operación en localhost; en producción se recomienda desplegar una instancia gestionada o un servidor MongoDB securizado, con usuarios, copias de seguridad y políticas de acceso configuradas.
- Modelo de IA único: el sistema está configurado para un único modelo local; una futura versión podría seleccionar entre varios modelos según el tipo de consulta.
- Autenticación administrativa básica: el panel dispone de inicio de sesión con credenciales y token JWT; en producción debe endurecerse con rotación de secretos, gestión avanzada de usuarios y políticas de contraseña.
- Latencia de inferencia: los tiempos de respuesta (2-8 segundos) son aceptables para un chatbot, pero superiores a servicios cloud; pueden reducirse con hardware de mayor capacidad.

- Escalabilidad horizontal: la arquitectura está diseñada para un único negocio; la parametrización para múltiples establecimientos es una línea de evolución identificada.

RESUMEN DEL CAPÍTULO

Este capítulo ha establecido la base completa del proyecto siguiendo las cuatro etapas narrativas propuestas: (1) se ha descrito el escenario de partida —una peluquería local en Santander con necesidades de digitalización de su gestión de citas— y se ha cuantificado la problemática concreta (pérdida estimada del 30 % de tiempo productivo en gestión manual); (2) se ha revisado el estado del arte de las soluciones existentes (plataformas SaaS, WhatsApp, gestión manual), identificando sus limitaciones; (3) se ha justificado por qué ninguna solución comercial combina conversación en lenguaje natural, privacidad de datos y coste cero de IA; y (4) se ha presentado la propuesta concreta de este trabajo.

Se han definido la hipótesis de partida, el objetivo general, cuatro objetivos específicos alineados con los capítulos posteriores (Capítulos 2, 3, 4 y 5) y cuatro objetivos transversales (privacidad, robustez, usabilidad y mantenibilidad). Se ha descrito la metodología iterativa e incremental (Pressman y Maxim, 2020), el stack tecnológico completo y las consideraciones normativas en materia de protección de datos. Los capítulos siguientes desarrollan, respectivamente, los requisitos del sistema, el análisis y diseño de la solución, la presentación del producto implementado y la evaluación final de resultados.

CAPÍTULO 2. DISCIPLINA DE REQUISITOS

Este capítulo convierte el contexto presentado en el Capítulo 1 en un contrato de requisitos verificable. Se describen los objetos relevantes del dominio de la peluquería, los actores humanos, los límites del sistema y los casos de uso que guían el desarrollo de la plataforma Corte Perfecto.

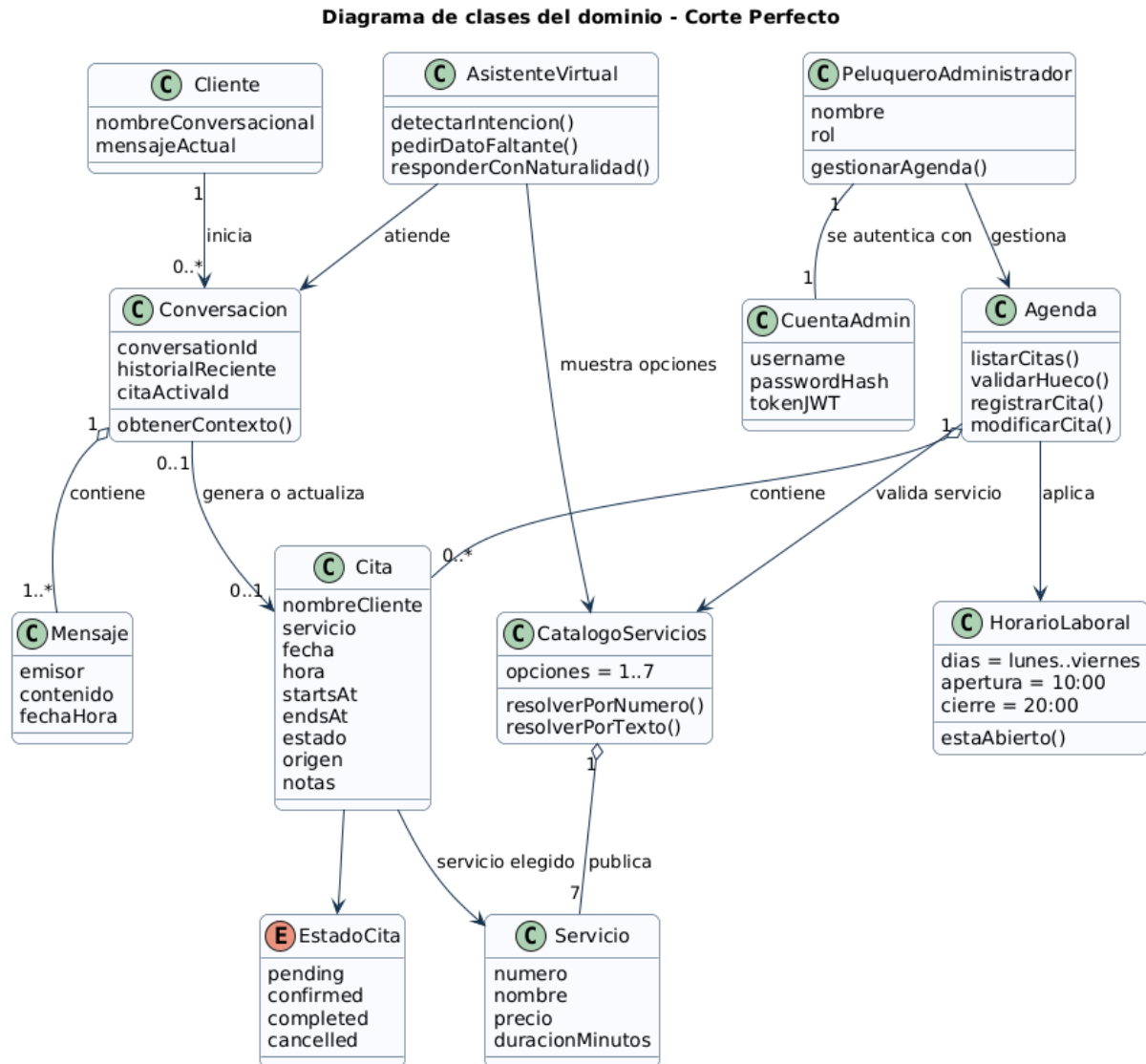
2.1 MODELO DEL DOMINIO

El modelo del dominio representa conceptos del negocio, se ha contrastado con el código actual para que las entidades descritas coincidan con la realidad de la aplicación: citas en MongoDB, catálogo numerado, chatbot local y panel de administración.

2.1.1 Diagrama de clases del dominio

Figura 2.1

Diagrama de clases del dominio de Corte Perfecto



El diagrama queda deliberadamente simple: el cliente inicia una conversación; la conversación genera o actualiza la cita; la agenda valida y contiene las citas; PeluqueroAdministrador gestiona la agenda; CuentaAdmin representa exclusivamente la parte de acceso y seguridad.

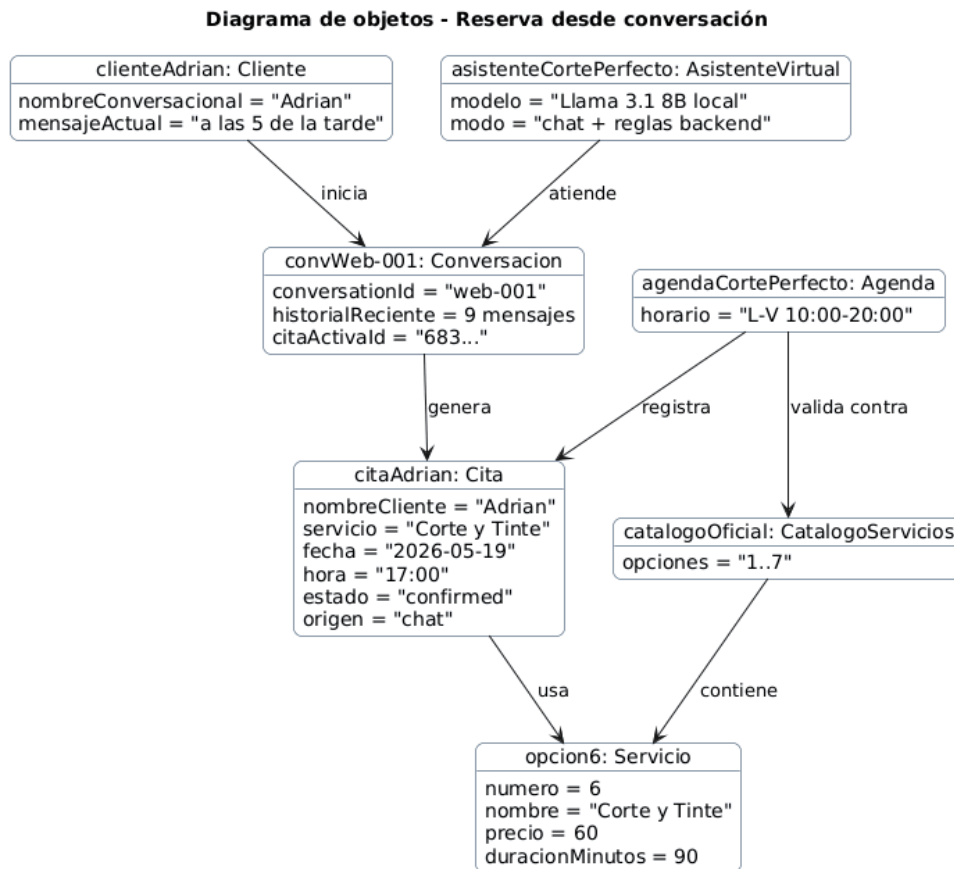
- Cliente: persona que conversa con el asistente y aporta los datos necesarios para reservar.
- Conversación: unidad de contexto del chatbot. Contiene mensajes y puede producir una cita cuando la información está completa.
- Cita: reserva persistida con nombreCliente, servicio, fecha, hora, intervalo, estado, origen y notas.
- Servicio y CatalogoServicios: fuente única de verdad para las siete opciones numeradas, precios y duraciones.
- Agenda y HorarioLaboral: concentran las reglas de disponibilidad, cierre de fin de semana y solapes.
- PeluqueroAdministrador y CuentaAdmin: se separa el rol operativo del peluquero de sus credenciales de acceso, respetando responsabilidad única.
- AsistenteVirtual: interfaz conversacional que atiende mensajes, pero no decide por sí sola las reglas críticas.

2.1.2 Diagrama de objetos

El diagrama de objetos muestra una reserva concreta realizada por chat. El administrador no aparece porque no participa en la creación conversacional de esa cita; su papel se modela en los casos de uso del panel privado.

Figura 2.2

Diagrama de objetos de una reserva desde conversación

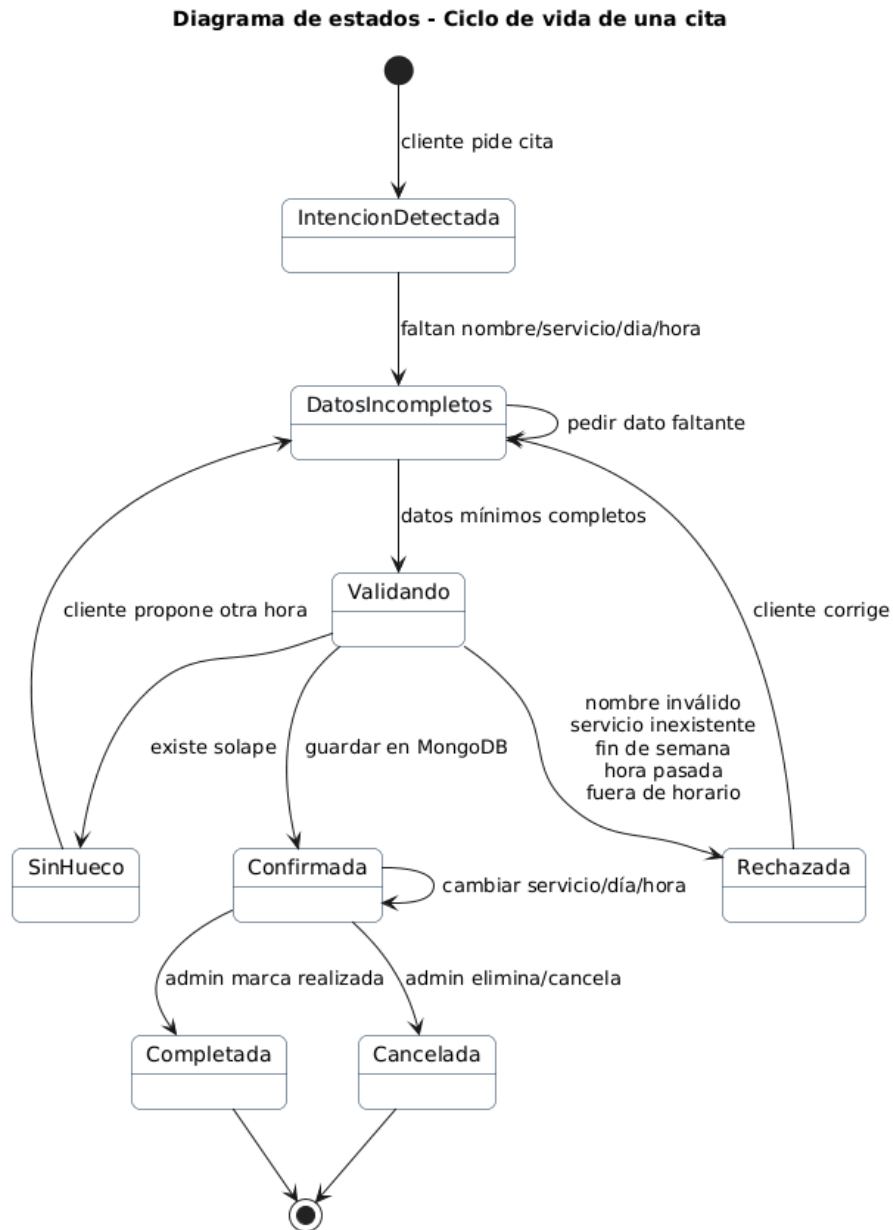


2.1.3 Diagrama de estados

El ciclo de vida refleja que una cita solo queda confirmada cuando el backend valida todos los datos y la persiste. Antes de eso existe una intención de reserva dentro de la conversación.

Figura 2.3

Estados principales de una cita



2.1.4 Requisitos suplementarios

Tabla 5

2.1.4 Requisitos suplementarios

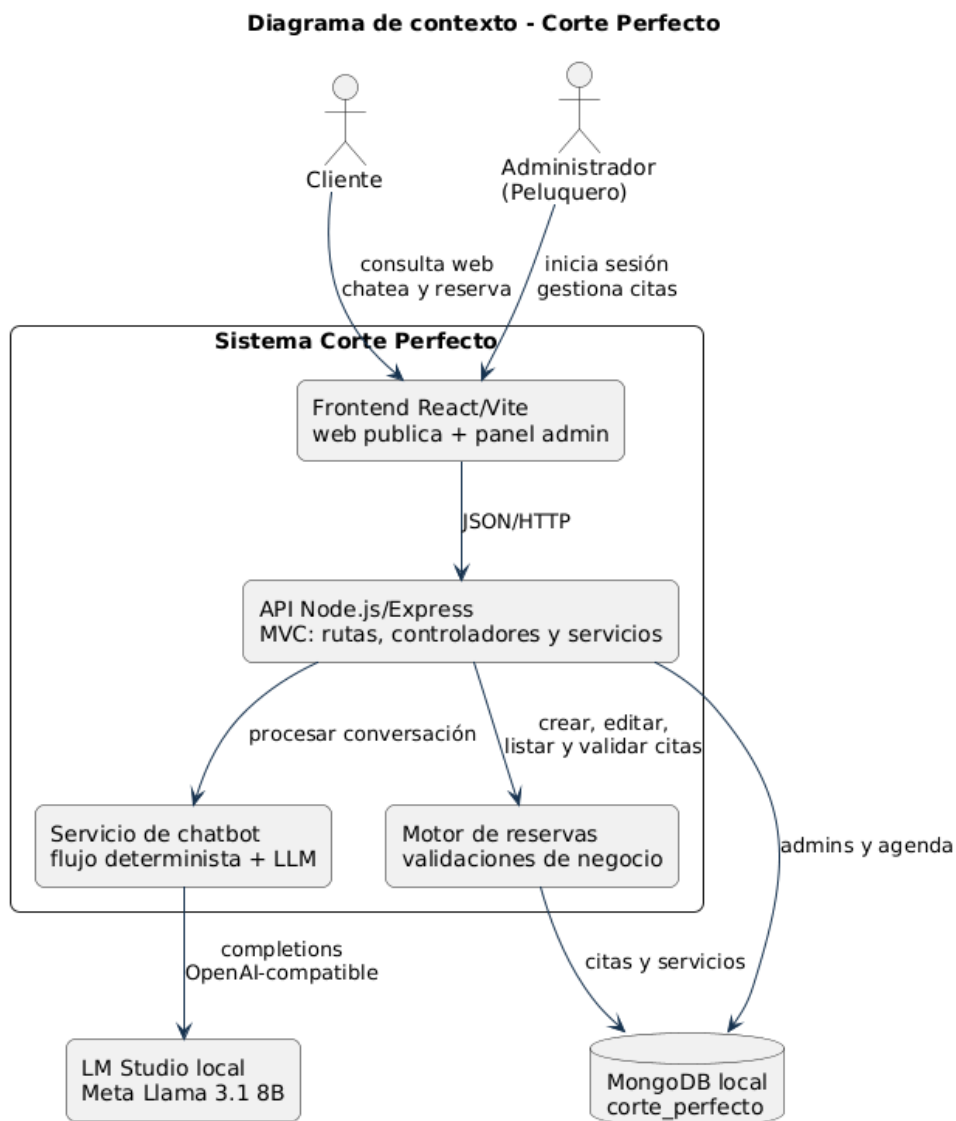
Código	Tipo	Descripción
RS-01	Rendimiento	Las operaciones REST sin inferencia LLM deben responder en menos de 1 segundo bajo carga local normal.
RS-02	Rendimiento IA	Las respuestas del chatbot deben tener timeout controlado de 60 segundos frente a LM Studio.
RS-03	Privacidad	Los mensajes no se envían a servicios externos de IA; la inferencia se realiza localmente.
RS-04	Seguridad	Las rutas privadas de administración requieren token JWT válido.
RS-05	Seguridad	La contraseña del administrador se guarda exclusivamente como hash bcrypt.
RS-06	Integridad	No se aceptan nombre inválido, servicio inexistente, fin de semana, hora pasada o fuera de horario.
RS-07	Integridad	No se permiten solapes entre citas activas.
RS-08	Usabilidad	La web y el chat deben funcionar en móvil y escritorio sin instalación.
RS-09	Usabilidad	Los servicios deben poder seleccionarse por número del 1 al 7.
RS-10	Compatibilidad	La aplicación debe funcionar en navegadores modernos.
RS-11	Mantenibilidad	El backend mantiene separación MVC y servicios con responsabilidades específicas.
RS-12	Extensibilidad	Añadir servicios debe centralizarse en el catálogo y no duplicarse en múltiples capas.
RS-13	Disponibilidad local	La ejecución en localhost requiere Node.js, MongoDB y LM Studio activos.
RS-14	Trazabilidad	Cada cita registra timestamps y origen chat/admin.
RS-15	Robustez	Si LM Studio falla, la API responde de forma controlada sin romper la interfaz.

2.2 DIAGRAMA DE CONTEXTO

El contexto conserva únicamente dos actores humanos: Cliente y Administrador/Peluquero. MongoDB y LM Studio no son actores de negocio, sino sistemas externos/locales con los que se integra la aplicación.

Figura 2.4

Diagrama de contexto del sistema



2.3 ACTORES DEL SISTEMA

Tabla 6

2.3 Actores del sistema

Actor	Tipo	Descripción
Cliente	Principal	Consulta la web, pregunta por horarios o servicios, conversa con el chatbot, elige servicio por número y solicita o modifica una reserva activa.
Administrador Peluquero	/ Principal	Inicia sesión en el panel privado, revisa estadísticas, filtra citas, crea reservas manuales, edita, marca como completadas, elimina citas y cierra sesión.

2.4 CASOS DE USO

2.4.1 Diagrama de casos de uso

Para mantener legibilidad, los casos de uso se separan por actor humano. Esta decisión evita un diagrama único demasiado denso y mantiene claro qué responsabilidades corresponden al cliente y cuáles al peluquero.

Figura 2.5

Casos de uso del cliente

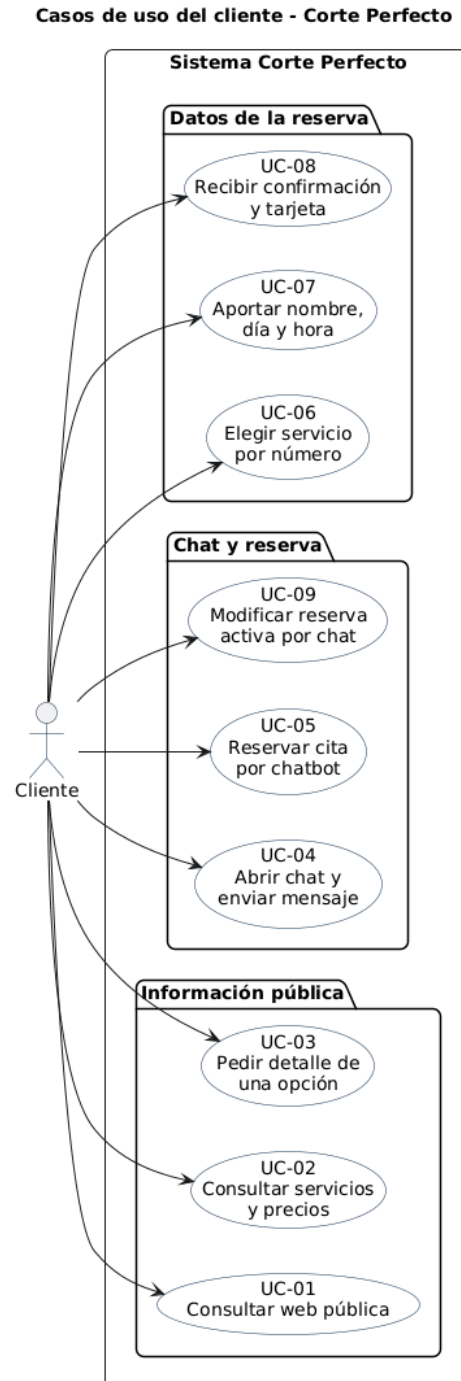
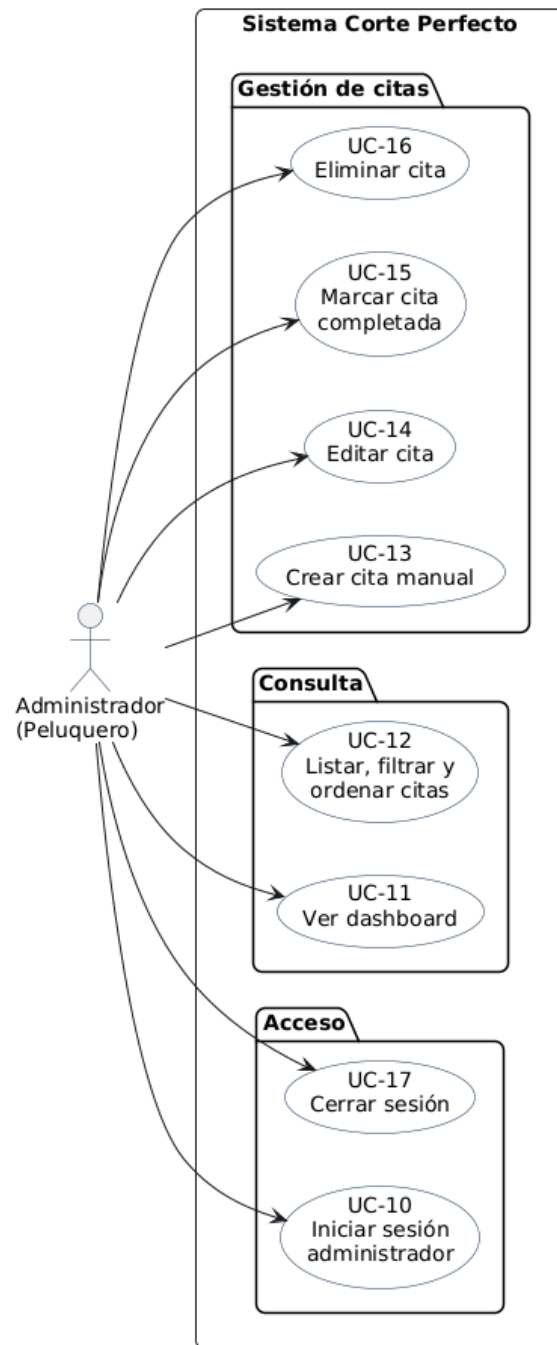


Figura 2.6

Casos de uso del administrador

Casos de uso del administrador - Corte Perfecto



2.4.2 Priorización de casos de uso

Tabla 7

2.4.2 Priorización de casos de uso

Código	Caso de uso	Actor principal	Prioridad
UC-01	Consultar web pública	Cliente	Media
UC-02	Consultar servicios y precios	Cliente	Alta
UC-03	Pedir detalle de una opción	Cliente	Media
UC-04	Abrir chat y enviar mensaje	Cliente	Alta
UC-05	Reservar cita por chatbot	Cliente	Alta
UC-06	Elegir servicio por número	Cliente	Alta
UC-07	Aportar nombre, día y hora	Cliente	Alta
UC-08	Recibir confirmación y tarjeta	Cliente	Alta
UC-09	Modificar reserva activa por chat	Cliente	Media
UC-10	Iniciar sesión administrador	Administrador	Alta
UC-11	Ver dashboard	Administrador	Media
UC-12	Listar, filtrar y ordenar citas	Administrador	Alta
UC-13	Crear cita manual	Administrador	Alta
UC-14	Editar cita	Administrador	Alta
UC-15	Marcar cita completada	Administrador	Media
UC-16	Eliminar cita	Administrador	Media
UC-17	Cerrar sesión	Administrador	Media

2.4.3 Detalle de casos de uso

Las fichas completas de UC-01 a UC-17 se conservan en RUP/02-requisitos/especificacion-casos-uso.md, con actor, precondiciones, flujo, alternativas,

postcondición y correspondencia con el código. Para que la memoria sea autosuficiente sin repetir diecisiete fichas extensas, se detallan aquí los recorridos de mayor riesgo funcional.

UC-05/06/07/08 - Reserva mediante chatbot. El cliente abre el chat, elige un servicio del catálogo 1..7 y aporta nombre, día y hora. El backend comprueba nombre, servicio, fecha futura, jornada de lunes a viernes, horario de 10:00 a 20:00 y ausencia de solapes. Si todo es válido, crea la cita y devuelve la confirmación; si falla una regla, solicita una alternativa sin persistir datos incorrectos.

UC-09 - Modificar una reserva activa. Con una cita identificada por `activeAppointmentId`, el cliente solicita cambiar nombre, servicio, fecha u hora. El sistema pregunta el nuevo valor si falta, recalcula precio y duración cuando procede y vuelve a validar la agenda. La misma cita se actualiza únicamente si el nuevo estado es válido.

UC-10 - Iniciar sesión como administrador. El peluquero envía usuario y contraseña desde `/admin/login`. El backend busca la cuenta, compara el hash mediante `bcrypt` y, si las credenciales son correctas, firma un JWT. Un error de autenticación no crea sesión ni permite acceder a las rutas protegidas.

UC-12/13/14/15/16 - Gestión administrativa de citas. Con sesión válida, el administrador lista, filtra y ordena la agenda; puede crear una cita manual, editarla, marcarla como completada o eliminarla tras confirmación. Todas las altas y modificaciones reutilizan las mismas reglas de servicio, horario, fecha y solape que el chatbot, por lo que existe una única lógica de negocio.

2.4.4 Diagramas de comportamiento de casos críticos

Los flujos críticos son la reserva por chatbot, su confirmación técnica y la gestión administrativa. Estos diagramas reflejan la separación de responsabilidades: interfaz, controlador, servicio de conversación, servicio de citas y persistencia.

Figura 2.7

Actividad principal de reserva por chatbot

Actividad - Reserva de cita mediante chatbot

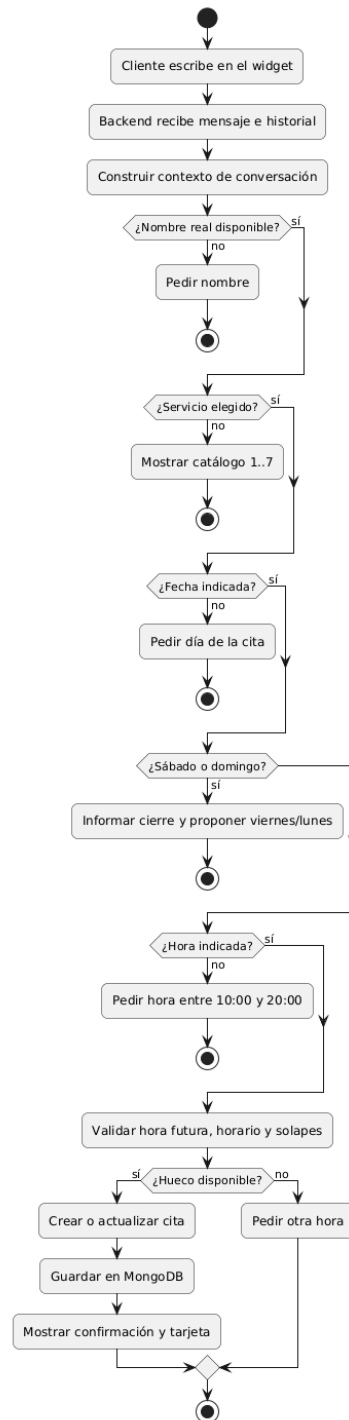


Figura 2.8

Secuencia técnica de confirmación de cita por chat

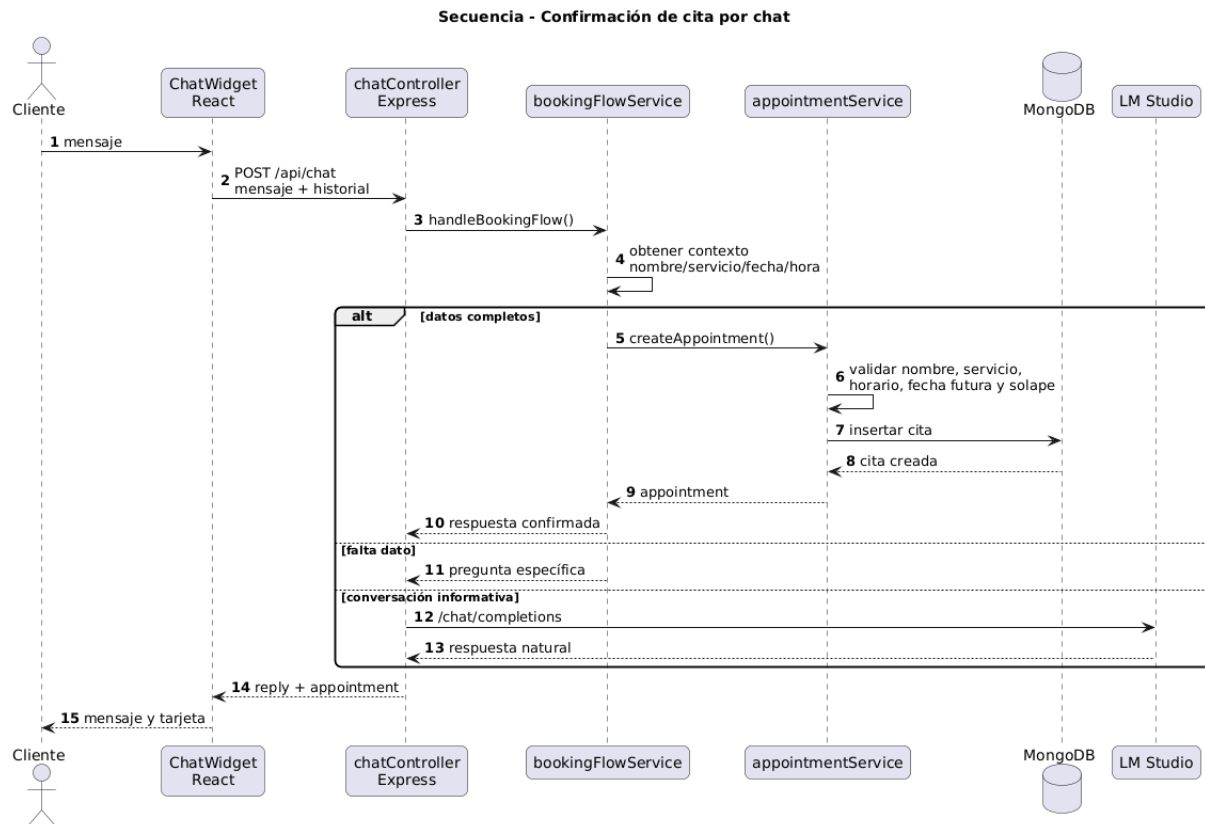
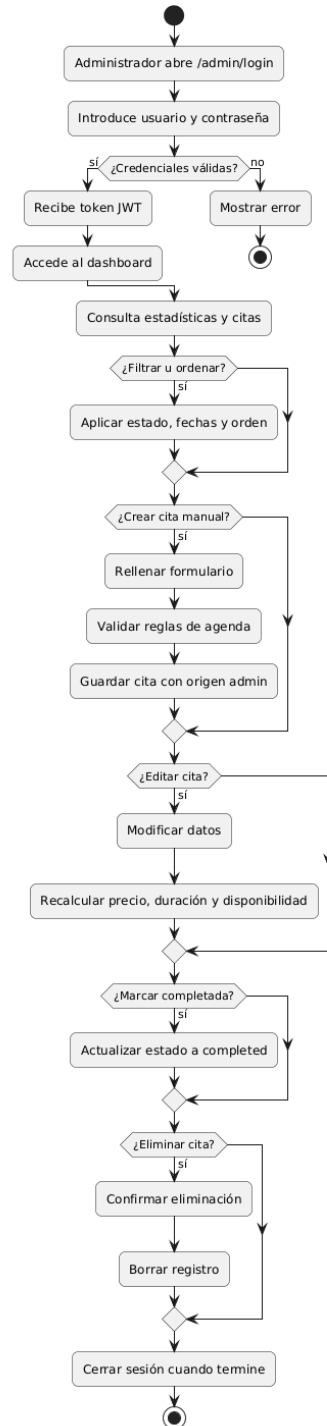


Figura 2.9

Actividad de gestión administrativa de citas

Actividad - Gestión administrativa de citas



2.4.5 Matriz de trazabilidad RS-UC

Tabla 8

2.4.5 Matriz de trazabilidad RS-UC

Requisito	Casos relacionados	Justificación
RS-01	UC-01, UC-02, UC-10, UC-12, UC-13, UC-14	Las operaciones CRUD y consultas dependen de respuesta rápida de API.
RS-02	UC-04, UC-05, UC-09	La conversación puede depender de LM Studio.
RS-03	UC-04, UC-05, UC-09	El chatbot trabaja con IA local.
RS-04	UC-10, UC-11, UC-12, UC-13, UC-14, UC-15, UC-16, UC-17	Todo el panel privado requiere JWT.
RS-05	UC-10	El login usa bcrypt.
RS-06	UC-05, UC-07, UC-13, UC-14	Toda reserva exige datos válidos.
RS-07	UC-05, UC-09, UC-13, UC-14	La agenda impide solapes.
RS-08	UC-01, UC-04, UC-12	La web y el chat deben ser cómodos en móvil y escritorio.
RS-09	UC-02, UC-03, UC-05, UC-06	El catálogo numerado reduce errores del chatbot.
RS-10	UC-01, UC-04, UC-10, UC-12	Las pantallas principales deben funcionar en navegadores modernos.
RS-11	Todos	MVC y servicios separados facilitan mantenimiento.
RS-12	UC-02, UC-03, UC-05, UC-13, UC-14	El catálogo central se reutiliza en web, chat y panel.
RS-13	Todos	La ejecución local requiere servicios activos.
RS-14	UC-05, UC-09, UC-12, UC-13, UC-14, UC-15	Origen y timestamps permiten auditar citas.

Requisito	Casos relacionados	Justificación
RS-15	UC-04, UC-05, UC-09	El fallo de LM Studio no debe romper la aplicación.

2.4.6 Prototipos de los casos de uso

Los prototipos se utilizaron durante el desarrollo para validar los recorridos del cliente y del administrador. Las pantallas finales y su correspondencia con los casos de uso se documentan en el Capítulo 4, evitando repetir aquí las mismas vistas.

2.4.7 Trazabilidad RUP y seguimiento de casos de uso

Además, se incorpora una trazabilidad viva que conecta requisitos, análisis, diseño, código y pruebas. Esta evidencia no sustituye a los diagramas del capítulo, sino que permite comprobar que cada caso de uso importante tiene reflejo en módulos reales del sistema.

2.4.8 Criterios de aceptación verificables

Los casos de uso críticos se cierran con criterios de aceptación ejecutables. De esta forma, la especificación deja de ser únicamente descriptiva y pasa a estar respaldada por comprobaciones automáticas.

Tabla 9

2.4.8 Criterios de aceptación verificables

Regla de negocio	Caso de uso	Evidencia
No registrar citas en sábado o domingo.	UC-05, UC-07, UC-13, UC-14	calendarService.test.js y appointmentService.test.js
No aceptar horas pasadas ni fuera de 10:00 a 20:00.	UC-05, UC-07, UC-13, UC-14	appointmentService.test.js
No aceptar nombres inválidos o genéricos.	UC-05, UC-07	bookingFlowService.test.js y appointmentService.test.js
No solapar citas activas.	UC-05, UC-09, UC-13, UC-14	appointmentService.test.js
Aceptar selección de servicios por número 1..7.	UC-03, UC-06	bookingFlowService.test.js

RESUMEN DEL CAPÍTULO

El capítulo deja definido el dominio y los requisitos de Corte Perfecto con una regla de diseño clara: el cliente reserva a través de la conversación, y la cita solo se confirma cuando el backend valida y persiste la información. Los actores humanos son Cliente y Administrador/Peluquero, mientras que MongoDB y LM Studio se tratan como sistemas de apoyo.

Esta base permite pasar al Capítulo 3 con una arquitectura más limpia: frontend React/Vite, backend Node.js/Express organizado en MVC, servicios de negocio con responsabilidad única, catálogo centralizado y persistencia MongoDB.

CAPÍTULO 3. ANÁLISIS Y DISEÑO

Este capítulo transforma los requisitos definidos en el capítulo anterior en una solución técnica concreta. La disciplina de análisis mantiene una abstracción cercana al problema, estructurada en arquitectura, casos de uso, clases y paquetes. La disciplina de diseño aterriza esa estructura en decisiones implementables: componentes React, rutas Express, servicios de negocio, esquemas Mongoose, integración con MongoDB y comunicación con LM Studio.

La decisión de diseño principal es mantener la simplicidad operativa: la aplicación se ejecuta en localhost, separa frontend y backend, conserva el patrón MVC en el servidor y encapsula las reglas críticas de agenda en servicios de negocio. Así, el modelo de IA ayuda a conversar, pero no decide por sí solo si una cita es válida.

3.1 DISCIPLINA DE ANÁLISIS

El análisis actúa como puente entre los requisitos y el diseño. Según el enfoque trabajado en IdSw, añade formalismo sin caer todavía en detalles accidentales de implementación. Por ello se describen clases de análisis, objetos de control, vistas y paquetes, prestando atención a cohesión, bajo acoplamiento, responsabilidad única y trazabilidad.

3.1.1 Análisis de la arquitectura

La primera decisión arquitectónica consiste en escoger un estilo que encaje con el tamaño del proyecto, el contexto local de ejecución y los requisitos suplementarios. En este TFG no se busca una plataforma multiempresa, sino una solución completa, comprensible y mantenible para una peluquería concreta.

Tabla 10

Comparación de estilos arquitectónicos

Estilo	Ventajas para Corte Perfecto		Limitaciones
Monolito tradicional	MVC	Despliegue sencillo y baja complejidad inicial.	Acopla presentación y lógica; dificulta evolucionar chat, panel y API por separado.
Microservicios		Escalabilidad y despliegues independientes por módulo.	Complejidad excesiva para un TFG y para una ejecución local con MongoDB y LM Studio.
Cliente-servidor con API REST		Separación clara entre React/Vite y Node.js/Express; facilita panel, chat e integración con IA local.	Requiere gestionar estado en cliente, tokens y errores HTTP.
Serverless		Menor gestión de infraestructura en nube.	No encaja con la inferencia local de LM Studio ni con el objetivo de localhost.

Se adopta una arquitectura cliente-servidor con API REST. El frontend React/Vite concentra la experiencia de usuario y el backend Node.js/Express concentra seguridad, validación, persistencia y coordinación con LM Studio. Esta elección se alinea con RS-03, RS-04, RS-06, RS-07, RS-11, RS-12 y RS-13.

En análisis conviene representar primero las capas conceptuales, porque es más fácil explicar responsabilidades y dependencias que con un diagrama técnico de componentes. Por ello la arquitectura se organiza en cuatro capas lógicas:

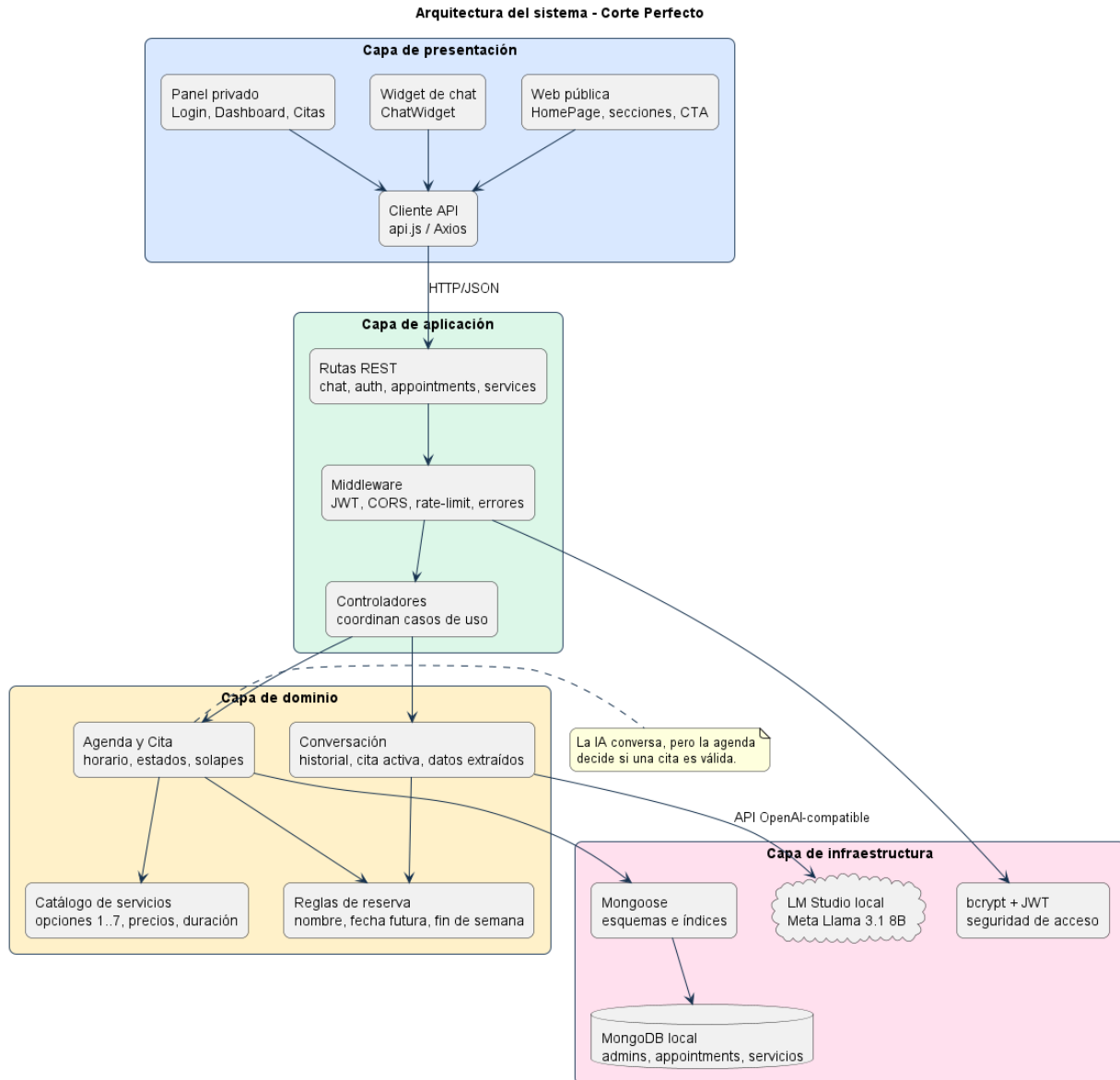
Tabla 11

Capas de la arquitectura

Capa	Responsabilidad	Tecnología / módulos
Presentación	Interfaz pública, widget de chat, login y panel administrativo.	React 19, Vite 7, React Router, componentes JSX, Axios.
Aplicación	Enrutado REST, controladores, seguridad, serialización JSON y coordinación de casos de uso.	Express 5, routes, controllers, middleware, JWT, rate-limit.
Dominio	Reglas de negocio de cita, catálogo, horario, solapes, estados y conversación.	appointmentService, bookingFlowService, calendarService, serviceCatalog.
Infraestructura	Persistencia documental, hash de contraseñas e inferencia local.	MongoDB, Mongoose, bcrypt, LM Studio API compatible con OpenAI.

Figura 3.1

Arquitectura lógica por capas de Corte Perfecto



La regla de dependencia es descendente: las vistas consumen casos de uso, los controladores coordinan, el dominio decide las reglas y la infraestructura adapta tecnologías externas. La IA local queda deliberadamente fuera del dominio: puede redactar una respuesta, pero la validez de la cita siempre la decide la agenda.

3.1.2 Análisis de casos de uso

No todos los casos de uso tienen el mismo peso arquitectónico. Para el análisis detallado se seleccionan los que obligan a tomar decisiones internas: consulta de catálogo numerado, reserva por chatbot, modificación de reserva activa, autenticación administrativa y gestión completa de citas. Cada uno se analiza con objetos de vista, control y modelo.

Tabla 12

3.1.2 Análisis de casos de uso

Caso	Objetos de análisis principales	Decisión de análisis
UC-02/03 Consultar servicios y detalle	VistaChat, VistaWebPublica, ConsultarServiciosController, CatalogoServicios, Servicio.	El catálogo 1..7 evita ambigüedad y permite responder por número sin interpretar una opción como fecha.
UC-05 Reservar cita por chatbot	VistaChat, ReservarCitaChatController, Conversacion, CatalogoServicios, Agenda, Cita.	El backend valida la reserva antes de persistirla; el LLM no es fuente de verdad.
UC-09 Modificar reserva activa	VistaChat, ModificarReservaChatController, Conversacion, Agenda, Cita.	Se reutiliza la cita activa y se recalculan servicio, precio, duración e intervalo.
UC-10 Iniciar sesión administrador	VistaLoginAdmin, LoginAdminController, CuentaAdmin.	La autenticación se separa del dominio de citas mediante hash bcrypt y token JWT.
UC-12/13/14/15/16 Gestión de citas	VistaGestionCitas, VistaFormularioCita, GestionarCitasController, Agenda, Cita.	El panel usa las mismas reglas de agenda que el chatbot para evitar duplicidad.

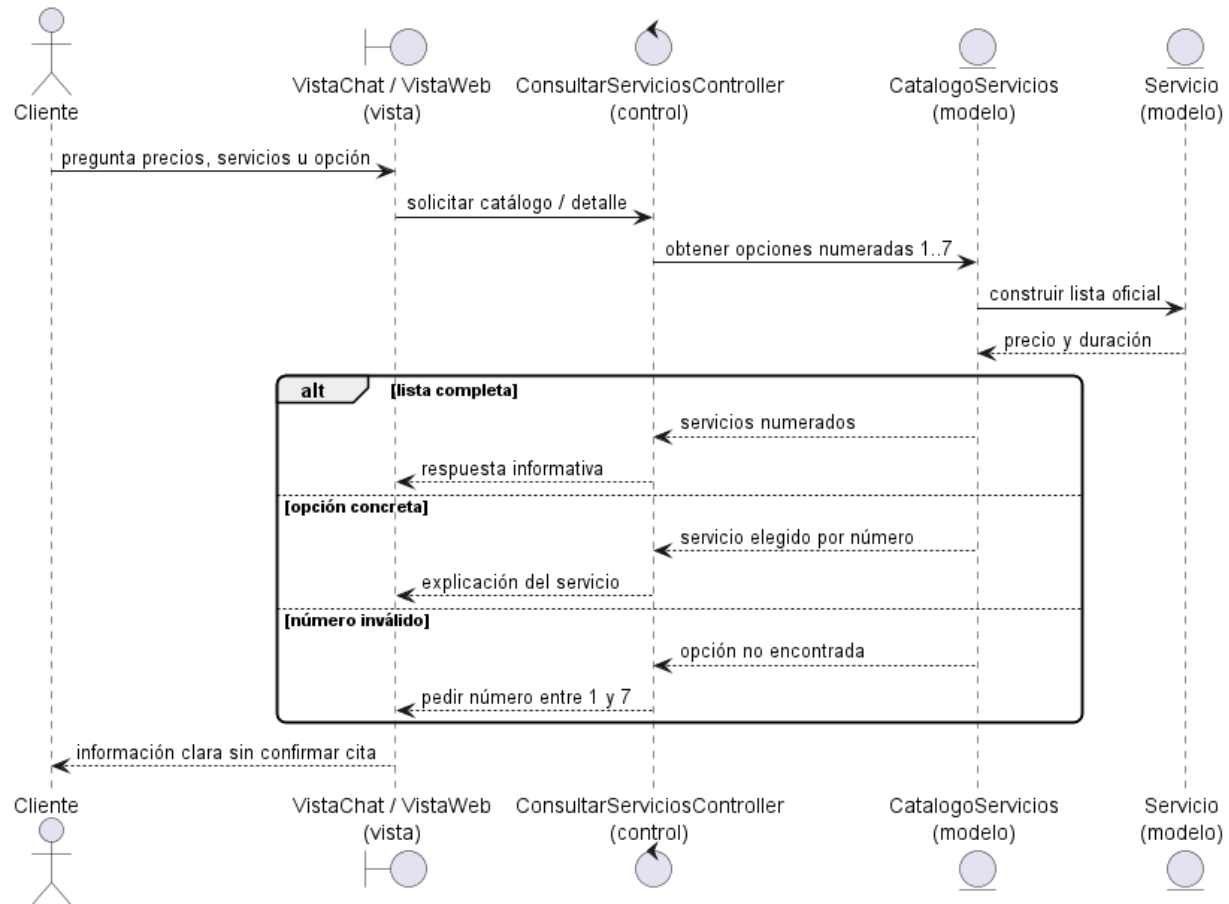
Análisis UC-02/03: consultar servicios, precios y detalle de opción

Este caso de uso es importante porque reduce errores conversacionales: el cliente puede pedir información general o preguntar por una opción concreta. El sistema no debe interpretar “opción 4” como una fecha, sino como “Corte y Peinado”.

1. El cliente pregunta por servicios, precios u opción concreta.
2. La vista envía la consulta al controlador o al flujo de reglas del chat.
3. El catálogo devuelve siempre las siete opciones oficiales con precio y duración.
4. Si el número está entre 1 y 7, se explica el servicio; si no, se repite la lista.
5. No se crea ninguna cita hasta que el cliente exprese intención clara de reservar.

Figura 3.2

Secuencia de consulta de servicios, precios y detalle de opción



Análisis UC-05: reservar cita por chatbot

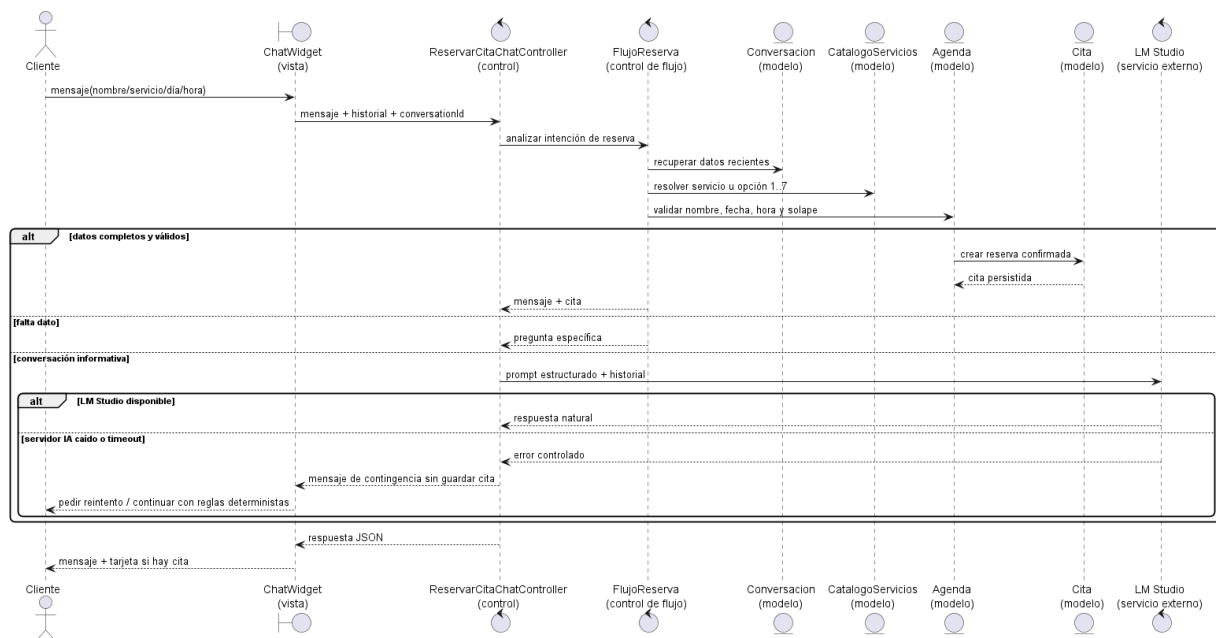
El flujo de reserva se analiza como una colaboración entre la vista conversacional, un controlador de chat, servicios de flujo y entidades de dominio. El cliente puede expresarse en lenguaje natural, pero la confirmación solo se produce cuando existen nombre, servicio, fecha y hora válidos.

1. El cliente escribe en el widget de chat.
2. La vista envía mensaje, historial y conversationId al controlador.
3. El controlador intenta resolver primero el flujo determinista de reserva.

4. El catálogo traduce números 1..7 y sinónimos a un servicio oficial.
5. La agenda valida nombre, horario, fecha futura, cierre de fin de semana y solapes.
6. Si todo es correcto, se crea la cita y se devuelve una tarjeta de confirmación.
7. Si LM Studio no responde, el sistema no confirma ni inventa una cita: devuelve un mensaje controlado de reintento y mantiene activa la validación determinista.

Figura 3.3

Secuencia de reserva de cita mediante chatbot



Análisis UC-09: modificar reserva activa por chat

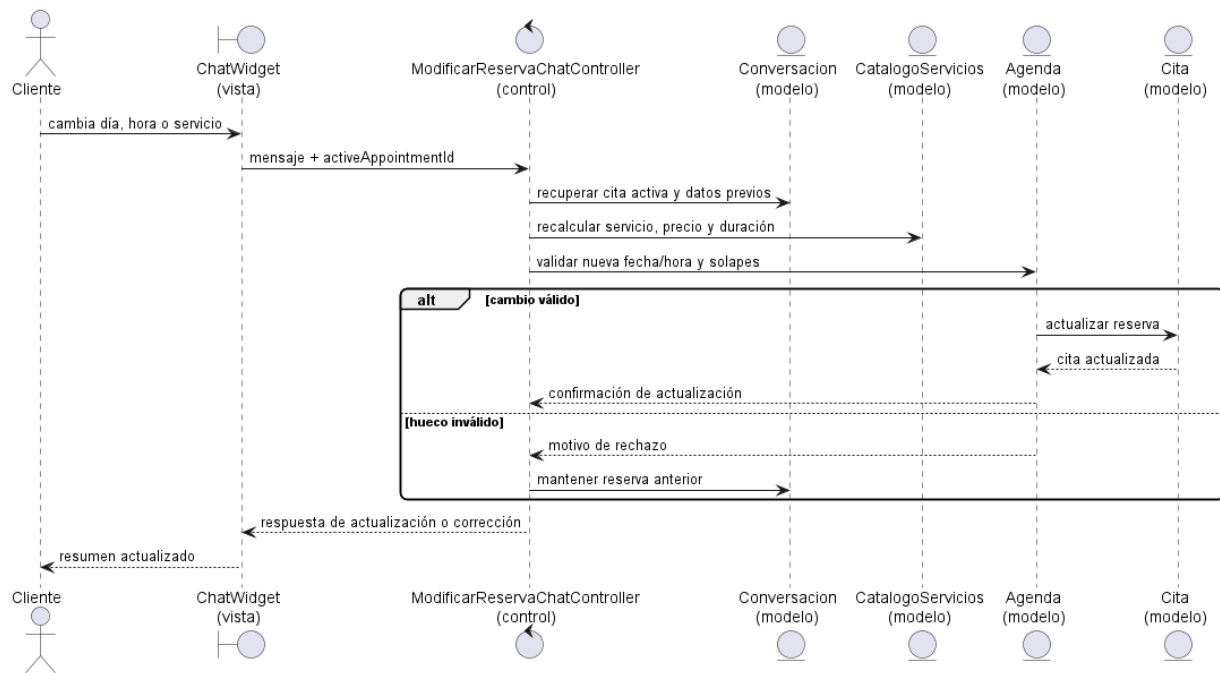
La modificación no se modela como una reserva nueva. Mientras el cliente no indique “otra cita” o “nueva reserva”, la conversación conserva una cita activa y aplica los cambios sobre ella.

1. El cliente solicita cambiar día, hora o servicio.
2. El controlador recupera la cita activa mediante `activeAppointmentId` y el contexto reciente.

3. El catálogo recalcula servicio, precio y duración si cambia el servicio.
4. La agenda valida de nuevo horario, fin de semana, fecha futura y solapes.
5. Si el cambio es válido, se actualiza la cita; si no, se conserva la reserva anterior.

Figura 3.4

Secuencia de modificación de una reserva activa por chat



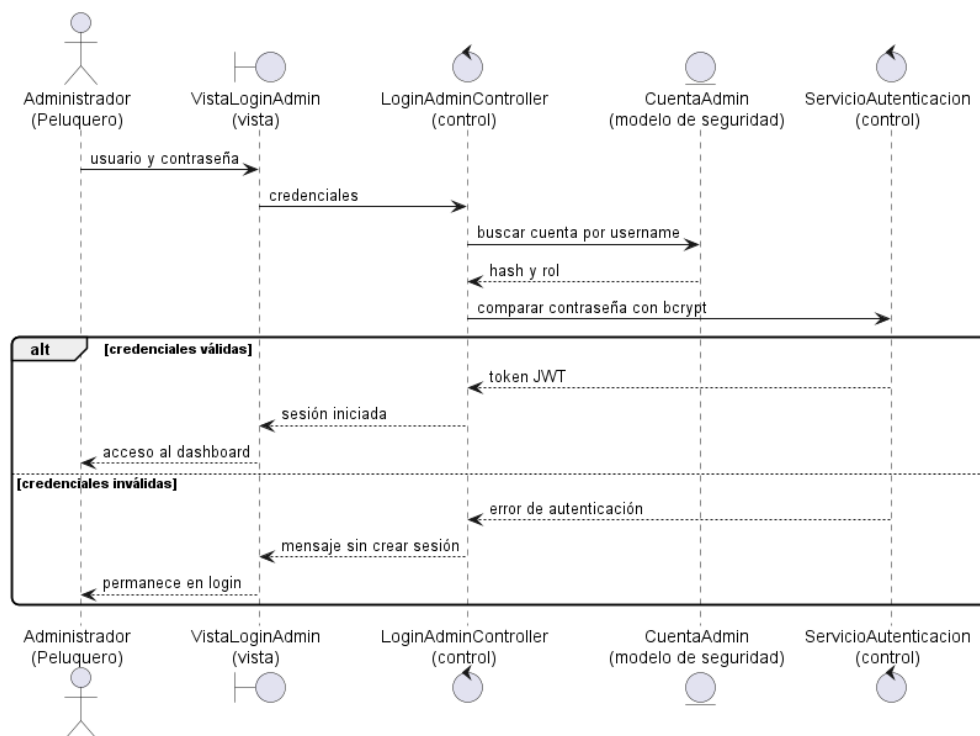
Análisis UC-10: iniciar sesión administrador

El administrador/peluquero es un actor humano, pero la cuenta administrativa se modela como una clase de seguridad. Su objetivo no es representar a una persona dentro del dominio de citas, sino autorizar el uso del panel privado.

1. El administrador introduce usuario y contraseña.
2. El controlador busca la cuenta por username.
3. La contraseña se compara con bcrypt frente al hash almacenado.
4. Si es correcta, se firma un JWT con identificador y rol.
5. Si es incorrecta, no se crea sesión y la vista muestra error.

Figura 3.5

Secuencia de inicio de sesión del administrador



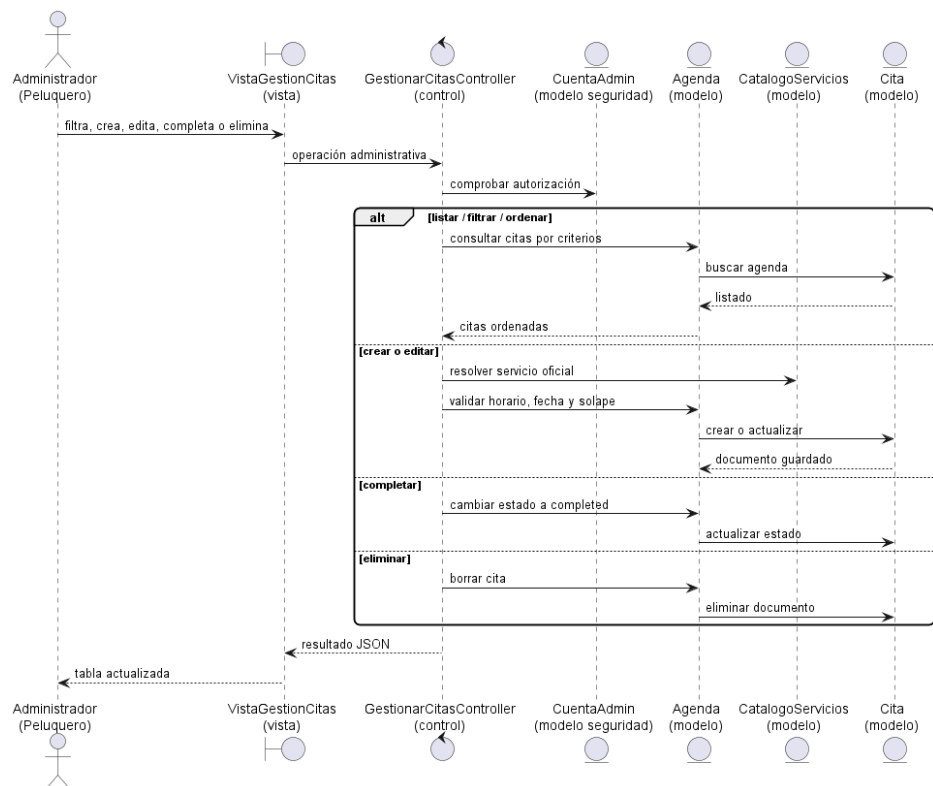
Análisis UC-12/13/14/15/16: gestión administrativa de citas

La gestión administrativa agrupa listar, filtrar, ordenar, crear, editar, completar y eliminar citas. Se analiza junta porque todas las variantes comparten el mismo actor, la misma vista privada y la misma entidad central: Agenda.

1. El administrador accede a Gestión de Citas con sesión válida.
2. El panel solicita listados filtrados por estado, fecha y orden.
3. Al crear o editar, se invocan las mismas reglas de horario, servicio y solape que usa el chatbot.
4. Al completar, se actualiza el estado a completed.
5. Al eliminar, se borra la cita tras confirmación de la acción.

Figura 3.6

Secuencia de gestión administrativa de citas

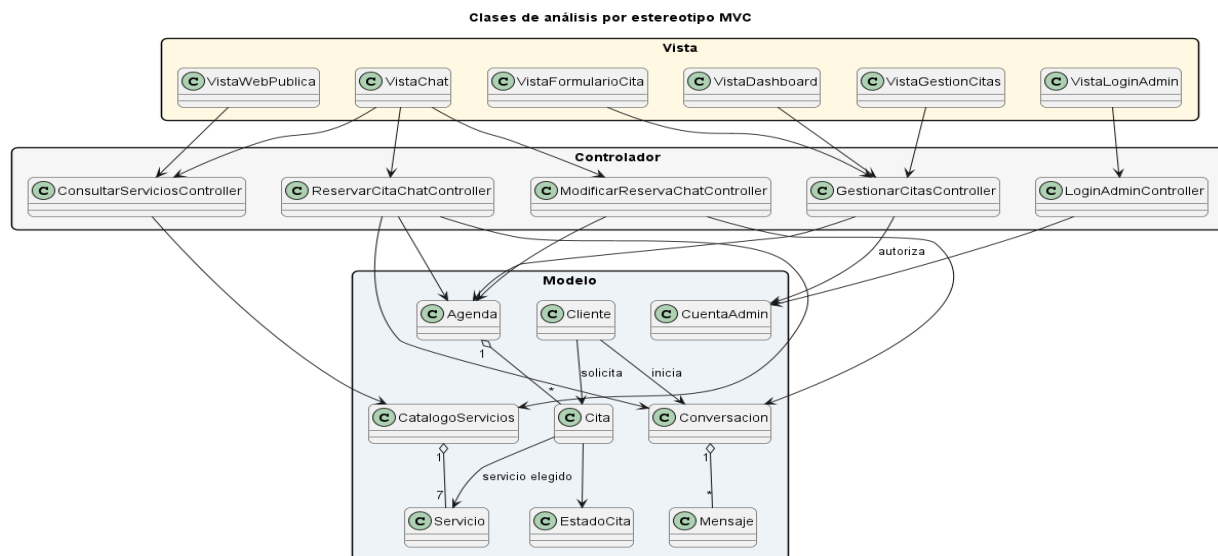


3.1.3 Análisis de clases

Partiendo del modelo de dominio del Capítulo 2 y de los casos de uso, las clases de análisis se clasifican en modelo, vista y controlador. Esta clasificación no es todavía código definitivo; sirve para repartir responsabilidades y detectar dependencias antes de diseñar los módulos reales.

Figura 3.7

Clases de análisis clasificadas por MVC



El diagrama distingue clases de vista, control y modelo. Las relaciones se limitan a las colaboraciones necesarias para ejecutar los casos de uso: las vistas invocan controladores, los controladores coordinan objetos de negocio y el modelo conserva las entidades conceptuales de conversación, agenda, cita, servicio y cuenta administrativa.

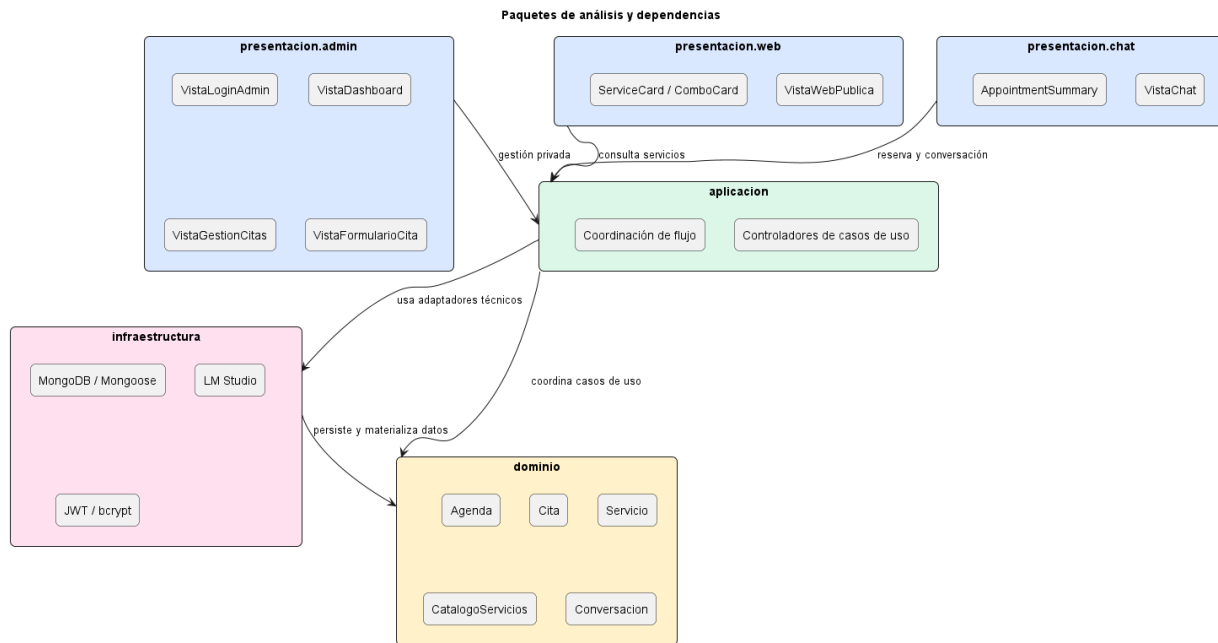
3.1.4 Análisis de paquetes

Los paquetes de análisis agrupan clases que cambian por el mismo motivo. Esta decisión responde a las directrices de cohesión y bajo acoplamiento: presentación reúne las

vistas, aplicación coordina los casos de uso, dominio concentra las reglas del negocio e infraestructura aporta persistencia, seguridad e integración local con IA.

Figura 3.8

Paquetes de análisis y dependencias



Las dependencias siguen una regla simple: presentación llama a aplicación, aplicación coordina el dominio e infraestructura da soporte técnico a los casos de uso. Así, cada paquete mantiene una responsabilidad clara y el esquema resulta más directo de explicar.

3.2 DISCIPLINA DE DISEÑO

El diseño convierte el modelo de análisis en una solución implementable. Aquí ya se nombran tecnologías, carpetas, módulos reales y mecanismos de comunicación. Se mantiene la trazabilidad con los requisitos suplementarios y con los casos de uso priorizados.

3.2.1 Diseño de la arquitectura

La arquitectura de diseño concreta la elección realizada en análisis. En este nivel ya aparecen los procesos reales, protocolos, rutas, middleware, módulos y servicios locales que permiten ejecutar la aplicación en localhost.

Figura 3.9

Arquitectura técnica de componentes y protocolos

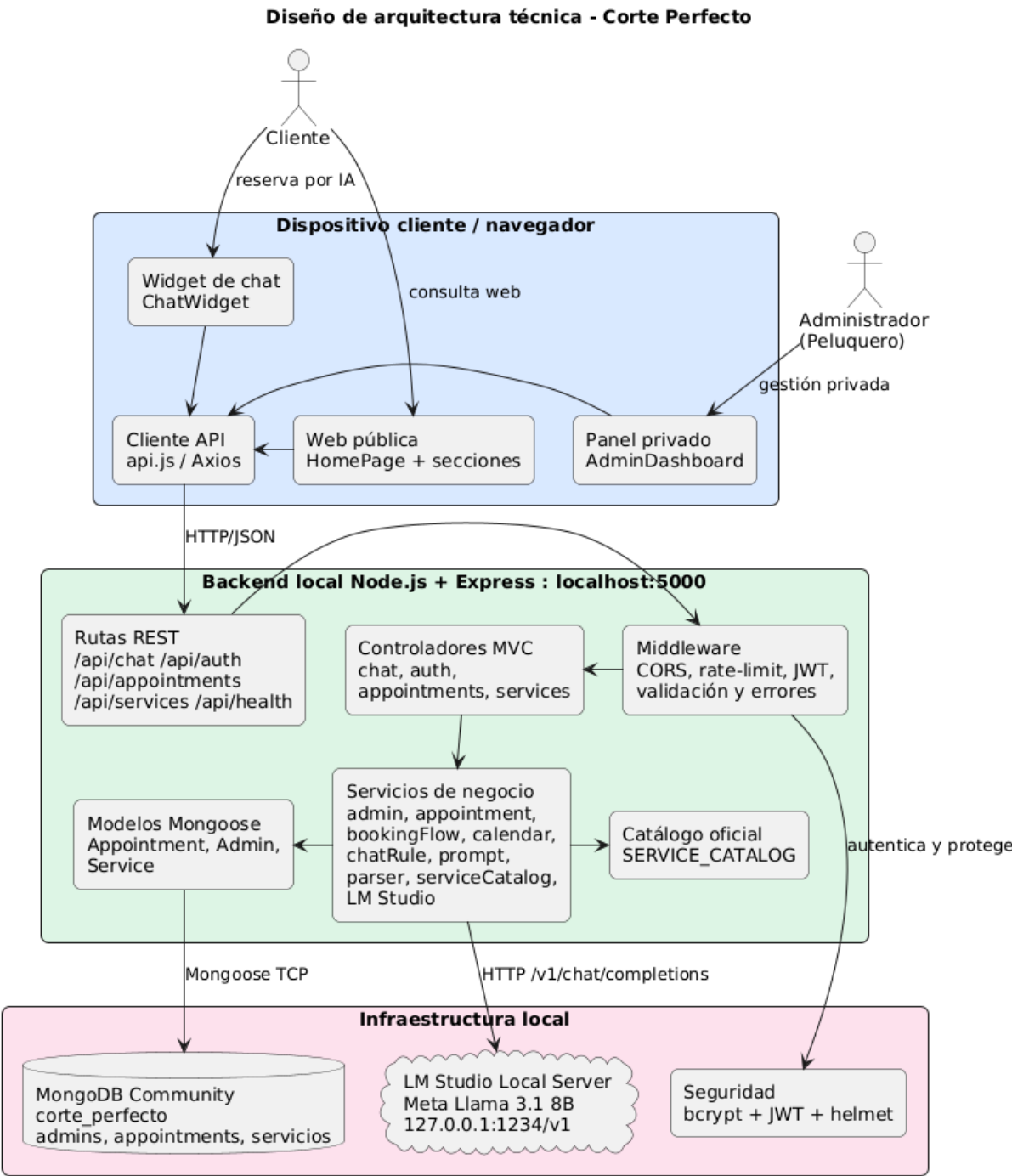


Tabla 13

Componentes de la arquitectura

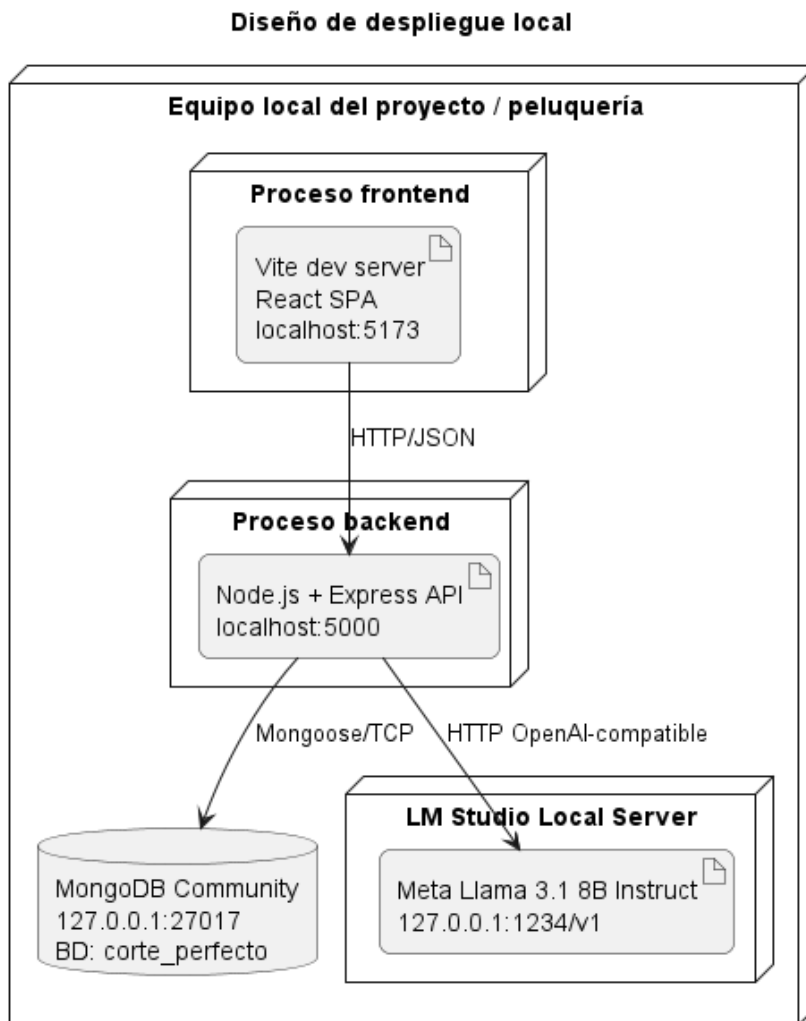
DESARROLLO DE UNA PLATAFORMA WEB INTEGRAL DE GESTIÓN DE CITAS PARA LA PELUQUERÍA CORTE
PERFECTO CON ASISTENCIA INTELIGENTE BASADA EN MODELOS DE LENGUAJE (LLM) DE EJECUCIÓN
LOCAL

Componente	Tecnología	Responsabilidad
Frontend	React 19 + Vite 7	Renderizar web pública, chat y panel privado; consumir API REST con Axios.
Backend	Node.js + Express 5	Exponer rutas REST, validar entradas, aplicar seguridad y coordinar servicios.
Base de datos	MongoDB + Mongoose	Persistir citas, administradores y catálogo de servicios sincronizado.
IA local	LM Studio + Meta Llama 3.1 8B	Generar respuestas conversacionales cuando el flujo determinista no basta.
Seguridad	JWT + bcrypt + helmet + CORS	Proteger panel privado, hash de contraseña y cabeceras HTTP.

El segundo esquema representa el despliegue local. No se añade una capa de contenedores porque no es necesaria para el objetivo del proyecto: el sistema se ejecuta en el equipo del desarrollador o de la peluquería con MongoDB y LM Studio abiertos localmente.

Figura 3.10

Diseño de despliegue local



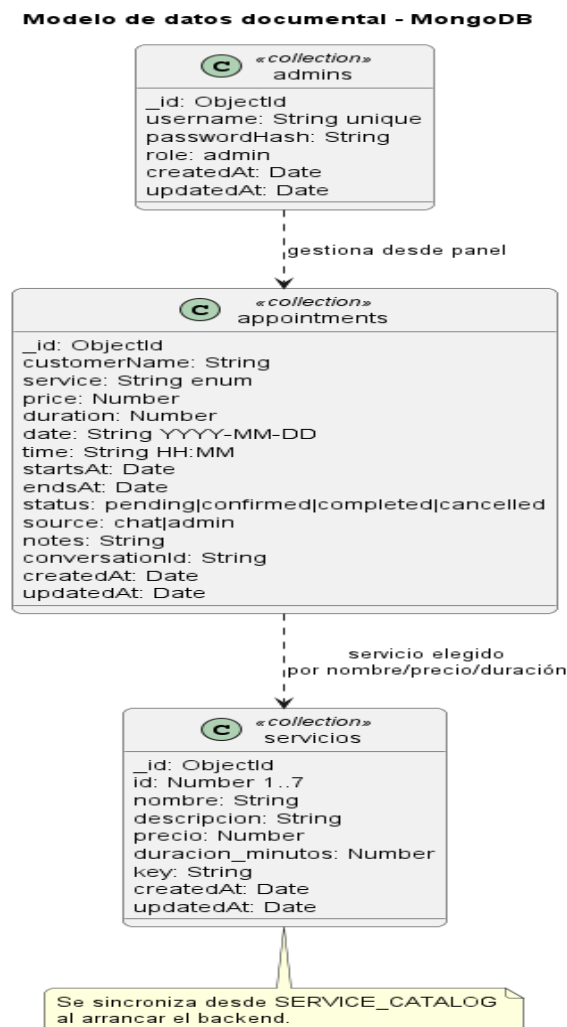
Autenticación y control de acceso: toda la API privada de citas está protegida con JSON Web Tokens. El middleware requireAuth verifica la firma del token, recupera la cuenta administrativa y rechaza peticiones sin credenciales válidas. El frontend conserva el token en localStorage y lo adjunta automáticamente mediante el interceptor de Axios.

3.2.2 Diseño del modelo de datos

MongoDB se utiliza como base documental porque el sistema maneja documentos de cita autocontenidos y necesita flexibilidad para registrar origen, notas, conversación y marcas temporales. Mongoose aporta validación de esquema, índices y un acceso homogéneo desde los servicios.

Figura 3.11

Modelo de datos documental en MongoDB



La base local se denomina corte_perfecto y las colecciones relevantes son appointments, admins y servicios. La colección servicios se sincroniza al arrancar el backend

desde SERVICE_CATALOG para que el catálogo numerado exista tanto en código como en MongoDB; las citas guardan servicio, precio y duración de forma desnormalizada para conservar el histórico aunque el catálogo cambie en el futuro.

Los índices de Appointment se diseñan alrededor de las consultas reales: startsAt/endsAt/status para agenda y solapes, y customerName/date/time/service para búsquedas y consistencia operativa.

3.2.3 Diseño de clases y módulos

En la implementación JavaScript, una clase de diseño puede materializarse como clase, esquema Mongoose, controlador Express, servicio exportado o módulo de configuración. La Figura 3.12 resume los módulos reales del backend que intervienen en la reserva y en la sincronización del catálogo. Las flechas discontinuas representan uso de módulos o paso de información; por eso ServiceCatalogService y ServiceModel aparecen separados del flujo conversacional: sincronizan y exponen el catálogo público de MongoDB, pero no participan en cada turno del chatbot.

Figura 3.12

Diseño simplificado de clases y módulos del backend de reserva

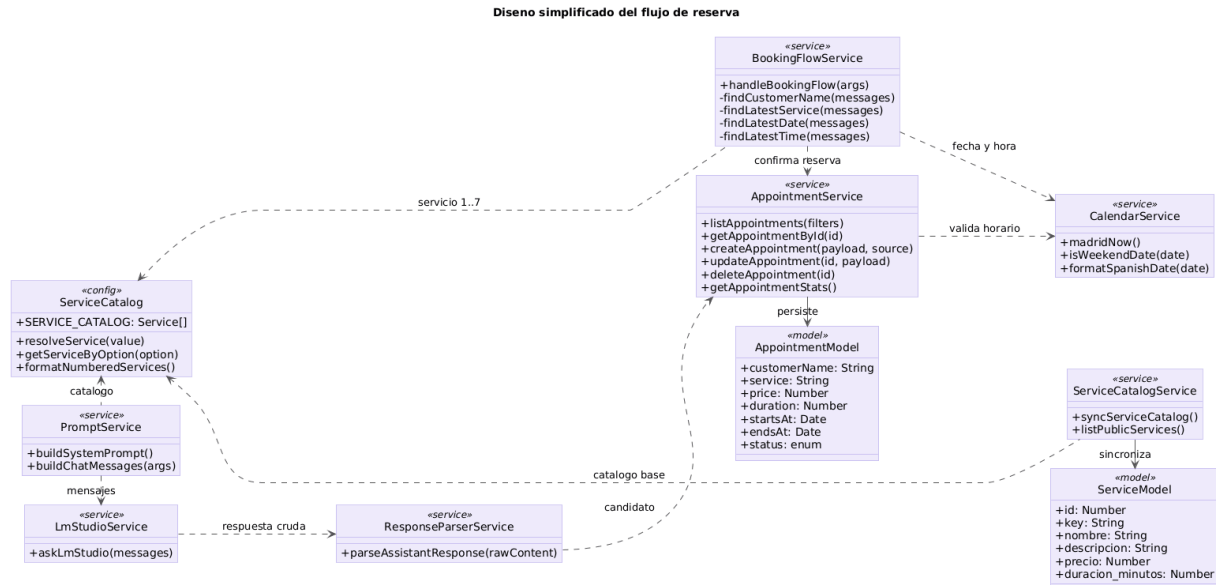


Tabla 14

Módulos y responsabilidades

Módulo	Responsabilidad	Principio aplicado
serviceCatalog.js / SERVICE_CATALOG	Define las siete opciones oficiales, resuelve servicios por texto u opción numérica y genera la lista numerada.	Fuente única de catálogo para prompt, validación y sincronización.
promptService	Construye el system prompt y los mensajes de chat con calendario, hora actual, historial y catálogo.	Separa la preparación del prompt respecto al controlador.
lmStudioService	Encapsula la llamada HTTP al servidor local de LM Studio, con timeout y error controlado.	Adaptador sustituible para el proveedor local de IA.
responseParserService	Separa la respuesta visible para el cliente del posible JSON de cita generado por la IA.	Aísla el parseo del lenguaje natural.

Módulo	Responsabilidad	Principio aplicado
bookingFlowService	Extrae nombre, servicio, fecha y hora del historial y confirma reservas deterministas cuando los datos están completos.	Responsabilidad única del flujo conversacional de reserva.
appointmentService	Lista, consulta, crea, actualiza, elimina y calcula estadísticas de citas; valida nombre, servicio, horario, fecha futura y solapes.	Alta cohesión: toda regla crítica de agenda está en un punto.
calendarService	Centraliza fecha/hora de Madrid, días laborables, fines de semana y formatos de fecha usados por chat y agenda.	Evita duplicar lógica temporal.
serviceCatalogService	Sincroniza SERVICE_CATALOG con la colección servicios y expone el catálogo público a la API.	Mantiene coherencia entre prompt, frontend y MongoDB.
adminService	Prepara la cuenta inicial, autentica al administrador y obtiene la sesión desde JWT.	Separa seguridad y credenciales de controladores y database.js.

Patrones y principios aplicados:

- MVC en backend: rutas y controladores forman la capa de entrada, servicios concentran negocio y modelos Mongoose representan persistencia.
- Fachada API en frontend: api.js centraliza baseURL, timeout, token JWT e interceptores.
- Adaptador para LM Studio: lmStudioService encapsula el endpoint compatible con OpenAI.
- Servicios de dominio: appointmentService, bookingFlowService y calendarService evitan controladores con lógica de negocio.
- Fuente única de catálogo: SERVICE_CATALOG alimenta prompt, validación y sincronización de la colección servicios.

3.2.4 Diseño de paquetes

La estructura de carpetas implementa los paquetes definidos en análisis y refleja el código real del repositorio. La dirección de dependencias es estable: las rutas llaman a controladores; los controladores delegan en servicios; los servicios usan modelos, configuración y utilidades; las vistas React consumen la API mediante una fachada HTTP centralizada. Además, el backend separa seed y tests, de modo que la inicialización del administrador y la verificación automática no quedan mezcladas con la lógica de ejecución.

TFG_CortePerfecto/

```
|— frontend/
|  |— src/
|     |— components/      # Brand, tarjetas, cabecera y ChatWidget
|     |— components/admin/ # AppointmentForm del panel privado
|     |— context/         # AuthContext y sesión JWT
|     |— data/            # Datos estáticos de servicios, combos y contacto
|     |— pages/           # HomePage pública
|     |— pages/admin/     # Login, layout, dashboard, citas y alta manual
|     |— services/        # api.js, fachada Axios
|     |— styles/          # CSS global
|     |— utils/           # Formateo de fechas y moneda
|— backend/
|  |— src/
|     |— config/          # env, database, serviceCatalog
|     |— controllers/     # auth, chat, appointments, services, health
```

		—	middleware/	# requireAuth y errores
		—	models/	# Admin, Appointment, Service
		—	routes/	# Rutas REST Express
		—	seed/	# Preparación del administrador inicial
		—	services/	# Negocio, IA local, catálogo y calendario
		—	utils/	# AppError, asyncHandler
		└—	app.js / server.js	# Configuración Express y arranque Node
		└—	tests/	# calendarService, bookingFlowService, appointmentService
		└—	diagramas/	# Fuentes PlantUML, capturas e imágenes
		└—	RUP/ y entregas/	# Trazabilidad y documentos finales

3.2.5 Diseño de interfaces de usuario

El diseño de interfaz se basó en dos recorridos: reserva conversacional del cliente y gestión autenticada del administrador. Las decisiones visuales y las pantallas implementadas se presentan en el Capítulo 4, junto con su validación funcional.

3.2.6 Diseño de integración con sistemas locales

La integración más sensible es el chatbot. La Figura 3.13 refleja el orden real de chatController: primero intenta bookingFlowService; si no hay respuesta determinista, consulta chatRuleService; cuando necesita IA, construye mensajes con promptService, llama a lmStudioService, parsea con responseParserService y solo después delega en appointmentService. Así ningún texto generado se persiste sin validación de negocio.

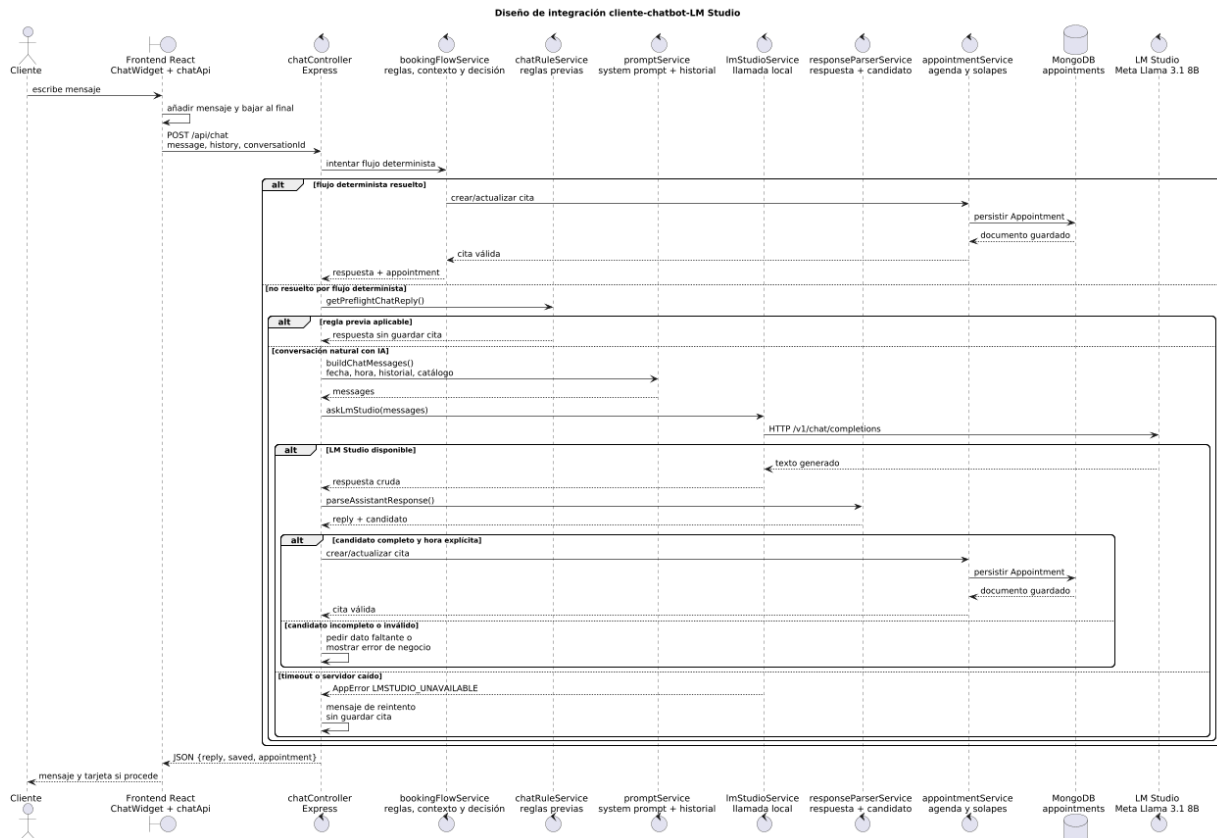
Tabla 15

3.2.6 Diseño de integración con sistemas locales

Sistema	Protocolo	Dirección	Control de errores
LM Studio	HTTP /v1/chat/completions	Backend -> LM Studio	Timeout de 60 s y respuesta 503 controlada.
MongoDB	TCP local mediante Mongoose	Backend MongoDB	-> Validación de esquema, índices y errores centralizados.
Frontend	HTTP/JSON con Axios	React -> Express	Timeout de cliente e interceptores para JWT y mensajes de error.
JWT	Bearer token	Frontend privadas -> rutas	Middleware requireAuth rechaza token ausente, inválido o caducado.

Figura 3.13

Integración del cliente con chatbot, backend, LM Studio y MongoDB



Este diagrama se centra en el cliente porque es la integración crítica de IA local. La integración administrativa ya queda cubierta por los diagramas UC-10 y UC-12/13/14/15/16: el peluquero no llama a LM Studio, sino que opera contra rutas privadas protegidas por JWT y contra la agenda persistida en MongoDB.

Si LM Studio está caído, el fallo no se propaga como una cita falsa. lmStudioService convierte el timeout o la conexión rechazada en un AppError controlado; el controlador devuelve un mensaje de reintento y el backend solo mantiene activos los flujos deterministas que no dependen de generación de lenguaje.

3.2.7 Estructura del system prompt y contingencia de IA local

El system prompt se diseña como una pieza de arquitectura, no como texto improvisado. Su objetivo es limitar la variabilidad del modelo, mantener el tono de peluquero real y delegar las decisiones críticas en reglas del backend. Por ello se construye dinámicamente desde promptService.js, incorporando calendario de Europe/Madrid, hora actual y catálogo numerado de servicios.

Tabla 16

3.2.7 Estructura del system prompt y contingencia de IA local

Bloque del prompt	Responsabilidad de diseño
Identidad y tono	Define al asistente como peluquero de Corte Perfecto, evita lenguaje técnico y mantiene una conversación natural.
Calendario fijo	Injecta fecha, hora actual, mañana, pasado mañana y próximos días laborables para evitar citas pasadas o fines de semana.
Prohibiciones	Impide pedir datos ya disponibles, confirmar sin nombre real, inventar disponibilidad o mostrar instrucciones internas.
Catálogo numerado	Publica las opciones 1..7 desde SERVICE_CATALOG para que cliente, prompt y backend compartan la misma fuente de verdad.
Reglas de reserva	Indica cómo pedir nombre, servicio, fecha y hora, y cómo emitir JSON solo cuando la información está completa.
Marcador de respuesta	Obliga a devolver el texto útil tras un marcador estable para que responseParserService pueda separar mensaje y JSON.

La caída de LM Studio se trata como una degradación controlada. En ese escenario el backend no crea citas basadas en una respuesta incompleta: conserva los flujos deterministas disponibles (servicios, horarios, fines de semana, citas pasadas y validación de

solapes) y devuelve un mensaje claro para que el usuario pueda reintentar cuando el servidor local vuelva a estar activo.

3.3 TRAZABILIDAD Y PLANIFICACIÓN TÉCNICA

La trazabilidad asegura que cada requisito suplementario tenga una decisión de diseño verificable. También sirve como guía de producción: primero se construyen los módulos de mayor riesgo técnico y después las vistas que dependen de ellos.

MEDIDAS DE CALIDAD DE DISEÑO

La calidad del diseño se evalúa mediante cinco criterios aplicables al alcance del proyecto: cohesión, acoplamiento, trazabilidad, mantenibilidad y testabilidad. La cohesión se considera adecuada cuando cada módulo concentra una responsabilidad reconocible; el acoplamiento se mantiene controlado cuando la comunicación entre capas se realiza mediante interfaces concretas; y la trazabilidad exige que requisitos, casos de uso, decisiones de diseño, implementación y pruebas puedan relacionarse sin contradicciones.

En el backend, estas medidas se reflejan en la separación entre rutas, controladores, servicios, modelos, middleware, configuración y utilidades. La lógica de calendario, reservas, catálogo, reglas conversacionales, preparación del prompt, comunicación con LM Studio y análisis de respuestas se distribuye en servicios especializados, evitando concentrarla en los controladores. En el frontend se distingue entre páginas, componentes reutilizables, contexto de autenticación, datos, utilidades y un único servicio de acceso a la API. Esta organización reduce dependencias directas y permite modificar una integración o una regla de negocio con un impacto localizado.

Como indicadores verificables, el repositorio contiene nueve servicios de backend con responsabilidades diferenciadas, tres modelos persistentes, cinco grupos de rutas y sus

controladores asociados, además de los componentes y páginas React necesarios para la web pública y la administración. La trazabilidad se documenta en RUP/99-seguimiento mediante el estado de los casos de uso y la auditoría diseño-implementación. La testabilidad se comprueba con 44 pruebas automatizadas superadas para calendario, flujo conversacional y gestión de citas, junto con la validación sintáctica del backend y la compilación correcta del frontend. Estos resultados no constituyen una certificación externa, pero aportan evidencia reproducible de que la estructura diseñada es coherente, mantenible y verificable.

3.4 AUDITORÍA DISEÑO-IMPLEMENTACIÓN

Siguiendo la práctica de revisión de pySigHor, se contrasta el diseño con el código real. El objetivo es evitar brechas entre lo modelado en análisis y diseño y lo construido finalmente en React, Node.js, Express, MongoDB y LM Studio.

3.5 PLAN DE PRUEBAS AUTOMATIZADAS

El plan de pruebas se centra en los riesgos principales del sistema: citas inválidas, solapes, errores de calendario, selección numérica de servicios y gestión administrativa de estados. Se priorizan pruebas de servicios porque ahí reside la lógica de negocio.

Tabla 17

3.5 Plan de pruebas automatizadas

Prueba	Riesgo cubierto	Módulos
calendarService.test.js	Interpretación incorrecta de días laborables y fines de semana.	calendarService
bookingFlowService.test.js	Pérdida de contexto conversacional, servicio numérico o fin de semana.	bookingFlowService, serviceCatalog
appointmentService.test.js	Citas inválidas, solapes, estados y borrado.	appointmentService, Appointment

Prueba	Riesgo cubierto	Módulos
npm run build --prefix frontend	Errores de integración o empaquetado del cliente.	React/Vite
npm run check --prefix backend	Errores sintácticos de entrada del servidor.	Node.js/Express

3.6 DASHBOARD RUP DEL PROYECTO

El seguimiento RUP se mantuvo en el repositorio para relacionar casos de uso, iteraciones y estado de implementación; la evidencia esencial queda recogida en las matrices de requisitos, diseño y pruebas incluidas en este documento.

Además de los diagramas del capítulo, el repositorio incluye un dashboard PlantUML de seguimiento. Este artefacto toma la idea de pySigHor de usar el diagrama de contexto como herramienta de gestión: los casos de uso no solo se dibujan, también se marca su estado de implementación y verificación.

El dashboard se conserva en RUP/99-seguimiento/estado-casos-uso.puml y permite enseñar de forma rápida qué casos están implementados por build y cuáles cuentan además con pruebas específicas.

RESUMEN DEL CAPÍTULO

Este capítulo convierte los requisitos de Corte Perfecto en un diseño implementable y trazable. El análisis define una arquitectura por capas, casos de uso críticos con sus colaboraciones, clases MVC y paquetes principales. El diseño concreta esas decisiones en React/Vite, Node.js/Express, servicios de negocio, MongoDB/Mongoose y LM Studio local. Con ello queda preparada una base clara para el capítulo de implementación, con especial atención a arquitectura, system prompt, secuencias críticas y contingencia ante caída de la IA local.

CAPÍTULO 4. IMPLEMENTACIÓN Y MAPA DE LA SOLUCIÓN

Este capítulo presenta la solución desarrollada desde el punto de vista de su materialización final. Después de haber definido requisitos, análisis y diseño en los capítulos anteriores, se muestra cómo esos artefactos se convierten en pantallas, flujos navegables, rutas de API, servicios de negocio y colecciones persistidas en MongoDB. El objetivo no es repetir el diseño técnico, sino demostrar que existe una aplicación funcional y coherente con el proceso metodológico seguido.

4.1 ESTADO TÉCNICO DE LA SOLUCIÓN IMPLEMENTADA

Antes de documentar la interfaz se revisó el estado del código para comprobar que la implementación respeta la arquitectura comprometida: frontend React/Vite, backend Node.js/Express organizado siguiendo MVC, persistencia MongoDB mediante Mongoose y conexión local con LM Studio. La comprobación ejecutada fue `npm run verify`, que encadena validación sintáctica del backend, pruebas automáticas y construcción de producción del frontend.

4.2 ORGANIZACIÓN FINAL DEL CÓDIGO

La organización del proyecto se mantiene deliberadamente simple. No se introducen microservicios ni capas accidentales porque el alcance del TFG se resuelve mejor con una API modular de alta cohesión. La separación entre frontend y backend es física y lógica: el frontend no accede a MongoDB ni a LM Studio; todo pasa por la API. A su vez, los controladores no concentran reglas de negocio, sino que delegan en servicios.

Tabla 18

4.2 Organización final del código

Capa	Carpetas / módulos	Responsabilidad
Presentación pública	frontend/src/pages, components, styles	Mostrar la web comercial, abrir el chatbot y renderizar la confirmación de cita.
Presentación administrativa	frontend/src/pages/admin, components/admin, context	Login, dashboard, filtros de agenda, formularios y acciones privadas.
Entrada backend	backend/src/routes, controllers, middleware	Exponer API REST, proteger rutas privadas, controlar errores y coordinar cada petición.
Dominio negocio	/ backend/src/services, config/serviceCatalog.js	Gestionar citas, autenticación, calendario, solapes, servicios, prompt, reglas de chat y respuesta de LM Studio.
Persistencia	backend/src/models, MongoDB	Persistir citas, cuenta administrativa y catálogo de servicios sincronizado.
Evidencia metodológica	RUP/99-seguimiento, backend/tests	Mantener trazabilidad caso de uso-código-prueba siguiendo el estilo de pySigHor.

Durante la revisión final se eliminaron responsabilidades mezcladas: la sincronización del catálogo de servicios queda centralizada en serviceCatalogService y la preparación/autenticación de la cuenta del administrador se ubica en adminService. Por tanto, database.js conserva una responsabilidad única: abrir la conexión con MongoDB. Esta decisión mejora la cohesión y evita que el módulo de conexión conozca reglas funcionales.

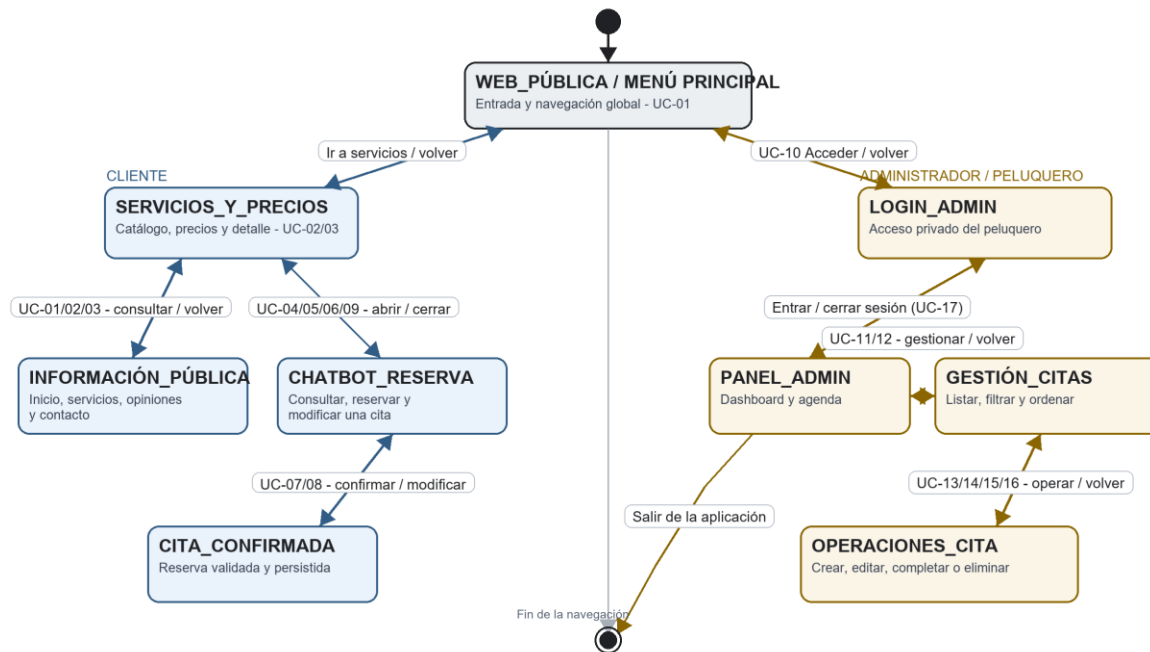
4.3 MAPA NAVEGABLE DE LA SOLUCIÓN

El mapa de navegación se deriva de los actores y casos de uso definidos en el Capítulo 2. El cliente entra en la web pública y puede consultar información o reservar mediante el chatbot. El administrador accede por una ruta privada, se autentica y gestiona la agenda.

Ambas experiencias convergen en la misma API y en la misma base de datos, lo que evita agendas paralelas.

Figura 4.1

Diagrama bidireccional de navegación por casos de uso



El diagrama adopta un enfoque de máquina de estados: parte de un contexto principal y representa las transiciones de navegación asociadas a UC-01–UC-17. La rama del cliente cubre la web pública, el chatbot y la confirmación de la cita; la rama administrativa cubre autenticación, dashboard, gestión, operaciones sobre citas y cierre de sesión. La integración técnica con la API, LM Studio y MongoDB se documenta en la Figura 3.13 y en los apartados 4.5 y 4.8.

Figura 4.2

Mapa navegable de la solución implementada

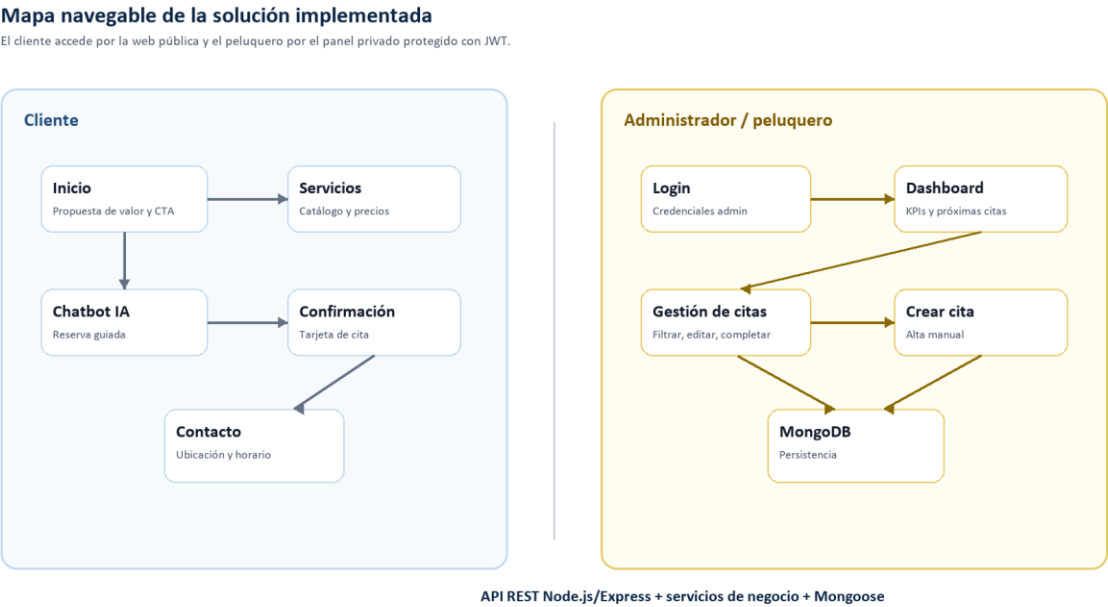


Tabla 19

4.3 Mapa navegable de la solución

Zona	Pantallas	Casos de uso cubiertos
Web pública	Inicio, Servicios, Combos, Nosotros, Opiniones, Contacto	UC-01, UC-02, UC-03
Chatbot	Widget flotante, historial, acciones rápidas, tarjeta de cita	UC-04, UC-05, UC-06, UC-07, UC-08, UC-09
Acceso admin	Login protegido	UC-10
Panel admin	Dashboard, Gestión de Citas, Crear Cita, Modal de edición	UC-11, UC-12, UC-13, UC-14, UC-15, UC-16, UC-17

4.4 WEB PÚBLICA: ESCAPARATE Y ENTRADA AL CHATBOT

La pantalla inicial concentra la identidad visual de Corte Perfecto, el acceso rápido a la reserva y un resumen de los servicios principales. La interfaz utiliza una estética oscura con acentos dorados, coherente con la percepción premium planteada desde el Capítulo 1. El botón principal abre el chatbot, por lo que la reserva no depende de que el cliente busque un formulario convencional.

Figura 4.3

Pantalla de inicio de Corte Perfecto



La sección de servicios materializa el catálogo oficial de la peluquería. En la web se presentan los servicios base y sus precios; en el chatbot se utiliza el mismo catálogo en formato numerado del 1 al 7 para reducir ambigüedades conversacionales. Esta decisión

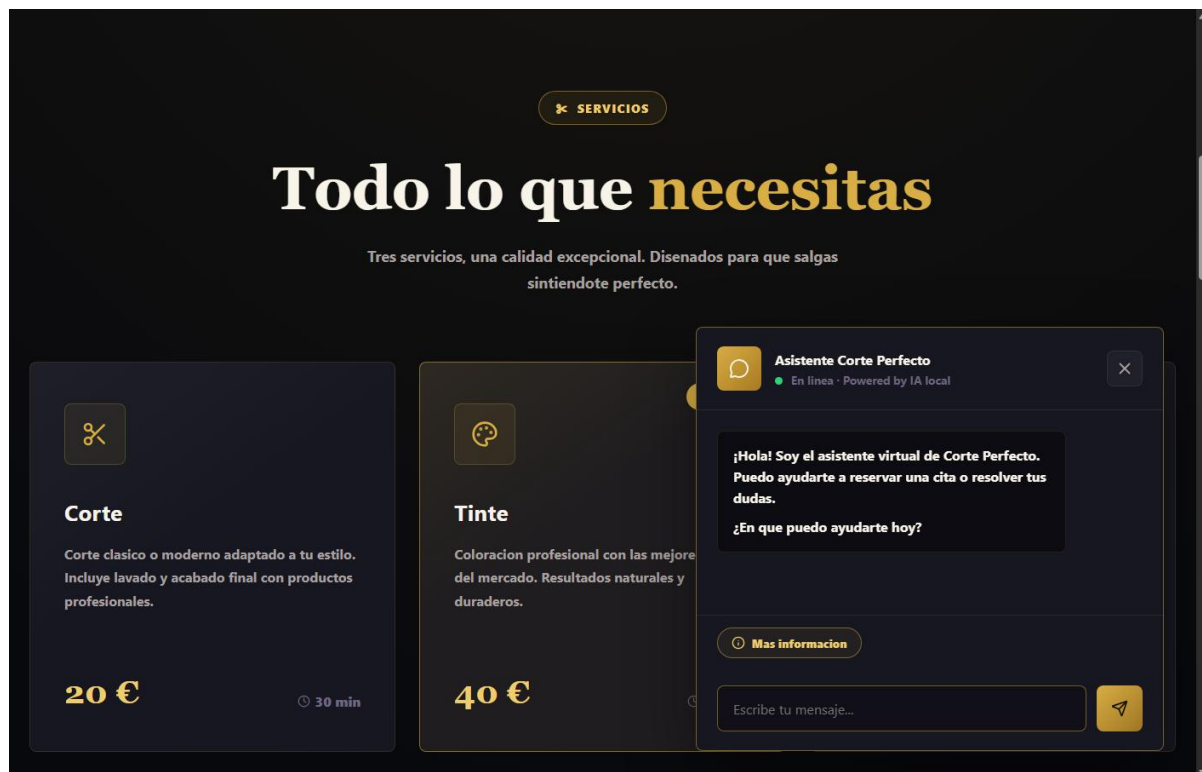
conecta directamente con la corrección realizada tras las pruebas iniciales: la opción numérica evita que el modelo interprete mal expresiones como “el 4”.

4.5 CHATBOT DE RESERVA CON LM STUDIO

El chatbot es el punto más singular del sistema. La interfaz se comporta como un asistente de reserva: saluda, ofrece información, acepta números de servicio, pregunta solo los datos que faltan y muestra una tarjeta cuando la cita se guarda correctamente. El componente ChatWidget mantiene el historial reciente y el identificador de conversación, pero la validación real se ejecuta en backend.

Figura 4.4

Widget de chat abierto sobre la web pública



La implementación evita confiar ciegamente en la salida del LLM. Primero se ejecutan reglas deterministas rápidas con chatRuleService; si el caso lo requiere,

promptService prepara el historial y lmStudioService consulta LM Studio mediante endpoint compatible con OpenAI; finalmente responseParserService separa el mensaje del posible candidato de cita y appointmentService lo valida antes de persistir. Así se mantiene una conversación natural sin trasladar a la IA decisiones críticas de agenda.

Figura 4.5

Flujo técnico de reserva por chatbot

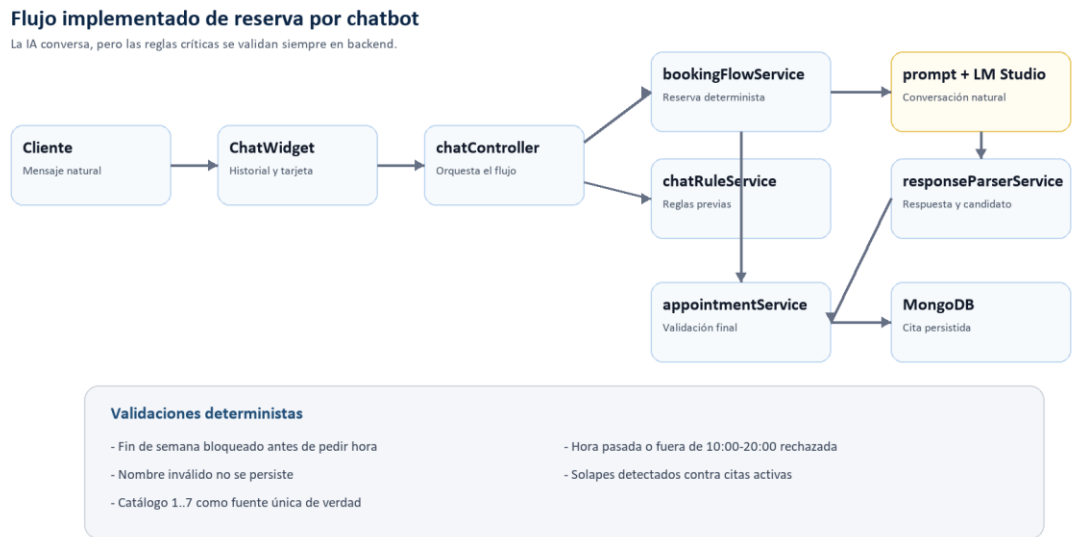


Tabla 20

4.5 Chatbot de reserva con LM Studio

Regla		Lugar de implementación	Efecto visible
Servicios número	por	serviceCatalog.js, chatRuleService.js, bookingFlowService.js	El cliente puede responder 1..7 y el sistema resuelve el servicio exacto.
Nombre obligatorio	real	bookingFlowService.js, appointmentService.js	No se registra una cita con nombre vacío, genérico o inválido.
Fin de semana cerrado	semana	calendarService.js, chatRuleService.js,	El sistema propone viernes o lunes antes de pedir hora.

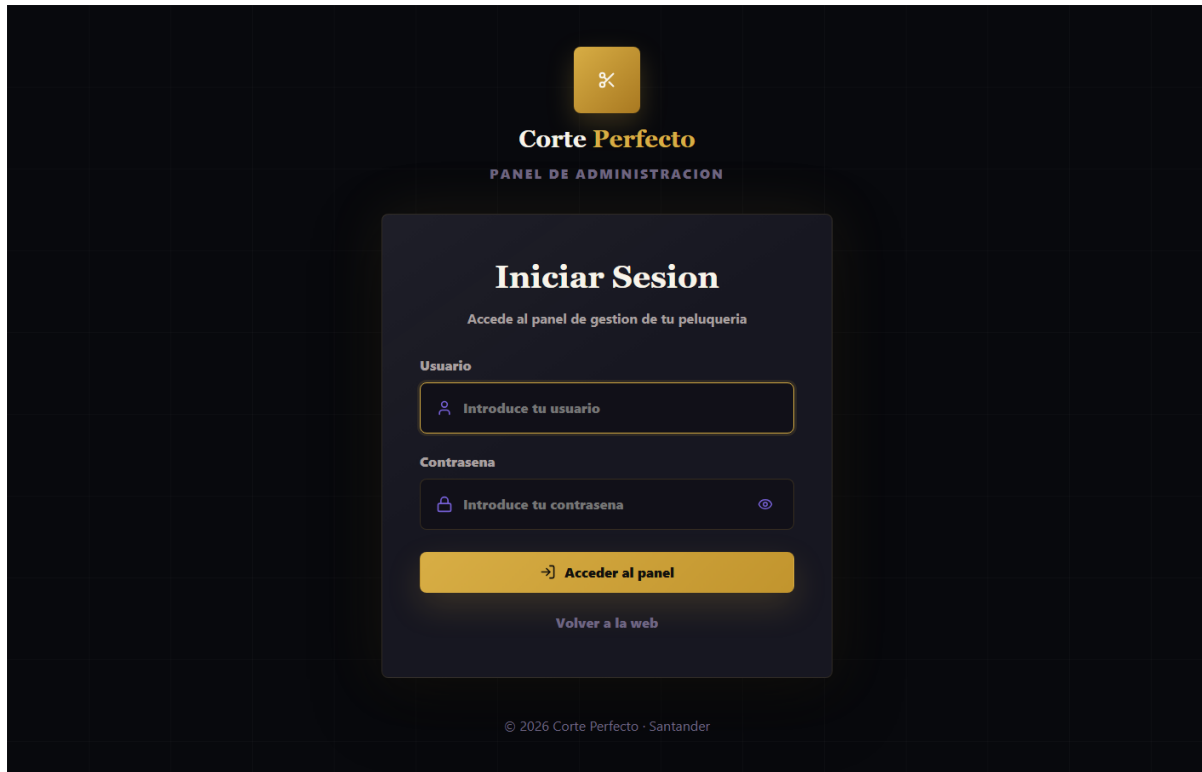
Regla	Lugar de implementación	Efecto visible
	bookingFlowService.js appointmentService.js	y
Hora actual	calendarService.js, chatRuleService.js appointmentService.js	y No se aceptan reservas para una hora pasada del mismo día.
Solapes	appointmentService.js	No se crean dos citas activas en el mismo intervalo.
LM Studio caído	lmStudioService.js, AppError.js errorMiddleware.js	y El usuario recibe un error controlado y no se inventa una confirmación.

4.6 ACCESO Y PANEL DE ADMINISTRACIÓN

El peluquero dispone de un panel privado. El acceso se realiza mediante usuario y contraseña; el backend valida la contraseña con hash y emite un JWT. Las rutas de citas quedan protegidas por middleware, de modo que un usuario sin token válido no puede consultar, modificar ni eliminar reservas.

Figura 4.6

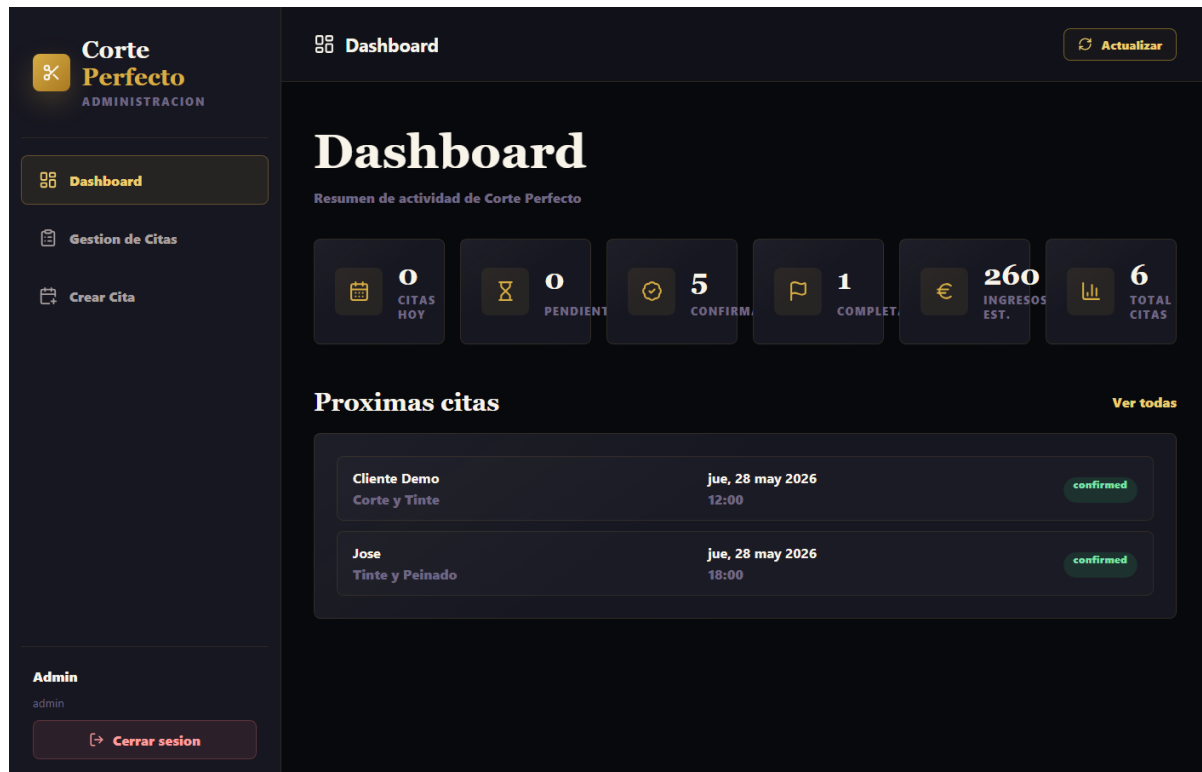
Pantalla de inicio de sesión del administrador



Tras iniciar sesión, el dashboard resume la actividad: citas del día, pendientes, confirmadas, completadas, ingresos estimados y total de citas. Esta pantalla no sustituye a la agenda detallada, pero ofrece al peluquero una lectura rápida del estado del negocio.

Figura 4.7

Dashboard administrativo



La gestión de citas permite filtrar por estado, acotar por fechas, ordenar cronológicamente y ejecutar acciones directas sobre cada reserva. Las acciones principales son editar, eliminar y marcar como completada. Esta última función fue incorporada para reflejar el ciclo de vida real de una cita: no basta con crearla, también debe poder cerrarse cuando el servicio ha sido prestado.

Figura 4.8

Gestión administrativa de citas



La creación manual cubre el escenario en el que el cliente llama por teléfono, acude presencialmente o el peluquero necesita registrar una reserva sin pasar por el chatbot. El formulario utiliza el mismo backend y las mismas reglas de validación que las citas conversacionales, por lo que la agenda mantiene una única fuente de verdad.

Figura 4.9

Formulario de creación manual de cita

4.7 CASOS DE USO REPRESENTATIVOS RESUELTOS EN INTERFAZ

La Tabla 21 enlaza los flujos representativos con su interfaz, endpoint, servicio y validación, demostrando la correspondencia entre modelado e implementación.

Tabla 21

Casos de uso representativos resueltos en interfaz

Casos	Interfaz y acción	API y módulo principal	Control verificable
UC-01/02/03	HomePage: información, catálogo y detalle.	GET /api/services; HomePage, ServiceCard y serviceCatalog.	Siete opciones coherentes en web y chat.
UC-04/05/06/07/08	ChatWidget: conversar, elegir servicio y reservar.	POST /api/chat; bookingFlowService y appointmentService.	Nombre, fecha, horario y solapes validados.

Casos	Interfaz y acción	API y módulo principal	Control verificable
UC-09	Modificar la reserva activa.	POST /api/chat; activeAppointmentId y updateAppointment.	Sin duplicados; se conserva si el cambio falla.
UC-10/17	Login y cierre de sesión.	POST /api/auth/login; adminService, AuthContext y api.js.	bcrypt, JWT, rutas privadas y control de 401.
UC-11/12	Dashboard, listado y filtros.	GET /api/appointments y /stats; vistas administrativas.	Filtros por estado, fechas y orden.
UC-13/14/15/16	Crear, editar, completar y eliminar.	POST/PATCH/DELETE; AppointmentForm y appointmentService.	Reglas compartidas y confirmación de borrado.

4.8 PERSISTENCIA Y SINCRONIZACIÓN DE DATOS

La base de datos corte_perfecto utiliza un modelo documental sencillo y suficiente para la primera versión del sistema. Las citas se guardan como documentos completos con nombre, servicio, precio, duración, fecha, hora, rango temporal, estado, origen y notas. El catálogo de servicios se sincroniza al arrancar desde una definición oficial del backend, lo que permite que MongoDB refleje las siete opciones disponibles sin convertir el catálogo en lógica duplicada.

Tabla 22

4.8 Persistencia y sincronización de datos

Colección	Uso	Campos principales
appointments	Agenda operativa de reservas.	customerName, service, price, duration, date, time, startsAt, endsAt, status, source, notes, conversationId.
admins	Cuenta de acceso del peluquero.	username, passwordHash, role, timestamps.
servicios	Catálogo público sincronizado.	id, key, nombre, descripcion, precio, duracion_minutos.

4.9 VERIFICACIÓN DE LA IMPLEMENTACIÓN

La verificación combina pruebas automáticas y comprobación manual de la interfaz. Siguiendo la filosofía de pySigHor, la evidencia no queda únicamente en el texto del TFG: también permanece en el repositorio mediante la carpeta RUP/99-seguimiento y las pruebas del backend.

Tabla 23

4.9 Verificación de la implementación

Comprobación	Comando / artefacto	Estado
Sintaxis backend	npm run check --prefix backend	Superado
Reglas de negocio	npm run test --prefix backend	44 pruebas superadas
Build frontend	npm run build --prefix frontend	Superado
Verificación integrada	npm run verify	Superado
Trazabilidad UC-código	RUP/99-seguimiento/trazabilidad-casos-uso.md	Actualizada
Auditoría diseño-implementación	RUP/99-seguimiento/auditoria-diseno-implementacion.md	Actualizada

4.10 RECORRIDO END-TO-END DE UNA RESERVA

El flujo completo de reserva confirma que la solución no funciona como piezas aisladas, sino como un circuito integrado. El cliente inicia la conversación en la web pública; el frontend envía el mensaje al backend; el backend decide si puede responder mediante reglas propias o si necesita consultar LM Studio; cuando se reúnen nombre, servicio, fecha y hora, la cita se valida y se guarda en MongoDB. Finalmente, el frontend muestra una tarjeta de confirmación y el panel administrativo puede consultar la misma reserva.

Tabla 24

4.10 Recorrido end-to-end de una reserva

Paso	Elemento responsable	Control aplicado
1. Inicio de conversación	ChatWidget.jsx	Crea identificador de conversación y conserva historial reciente.
2. Entrada al backend	chatController.js	Rechaza mensajes vacíos y coordina el flujo.
3. Reglas previas	chatRuleService.js	Responde a horarios/servicios, detecta fines de semana y opciones numéricas inválidas.
4. Flujo de reserva	bookingFlowService.js	Extrae nombre, servicio, fecha y hora manteniendo contexto conversacional.
5. IA local	lmStudioService.js	Solo se consulta cuando hace falta conversación abierta; timeout y error controlado.
6. Persistencia	appointmentService.js y Appointment.js	Valida nombre, servicio, horario, fecha futura y solapes antes de guardar.
7. Confirmación	ChatWidget.jsx	Muestra respuesta natural y tarjeta con datos persistidos.
8. Gestión posterior	AdminAppointments.jsx	Permite editar, completar o eliminar la misma cita.

4.11 REVISIÓN MVC Y PRINCIPIOS DE DISEÑO

El código no implementa SOLID de forma ceremonial con clases innecesarias; lo aplica en decisiones concretas: responsabilidades pequeñas, dependencias claras y servicios especializados. En un proyecto Node.js moderno, muchas unidades de diseño son módulos exportados en lugar de clases ES6, pero la separación conceptual se mantiene.

4.12 CIERRE DEL CAPÍTULO

La solución implementada cubre el flujo completo planteado al inicio del trabajo: un cliente puede conocer los servicios, conversar con un asistente local, reservar una cita y

recibir confirmación; el peluquero puede autenticarse, consultar la agenda, crear citas manualmente, modificarlas, completarlas o eliminarlas. La implementación conserva la separación de responsabilidades definida en el diseño y evita que la IA tenga control directo sobre la persistencia o sobre las reglas críticas de agenda.

CAPÍTULO 5. CONCLUSIONES Y LÍNEAS FUTURAS

El último capítulo valora el resultado obtenido. A diferencia del Capítulo 1, centrado en presentar el problema, este capítulo contrasta la hipótesis y los objetivos con la evidencia generada durante el desarrollo. También recoge las decisiones que han funcionado, las limitaciones razonables de una primera versión y las líneas de continuidad que permitirían evolucionar la plataforma.

5.1 CUMPLIMIENTO DEL OBJETIVO GENERAL

El objetivo general era diseñar e implementar una plataforma web de gestión de reservas para Corte Perfecto que automatizara la atención al cliente mediante un asistente conversacional local, persistiera las reservas en MongoDB y proporcionara una interfaz de administración. El resultado cumple ese objetivo: existe una web pública funcional, un chatbot conectado a LM Studio, una API Express con reglas de negocio, una base de datos MongoDB y un panel privado para el peluquero.

5.2 CUMPLIMIENTO DE OBJETIVOS ESPECÍFICOS

Tabla 25

5.2 Cumplimiento de objetivos específicos

Objetivo	Evidencia de cumplimiento	Valoración
OE1 Requisitos	Capítulo 2 define actores, casos de uso, modelo de dominio, estados, contexto y trazabilidad RS-UC.	Cumplido
OE2 Análisis y diseño	Capítulo 3 concreta arquitectura, MVC, paquetes, modelo documental, secuencias, prompt y contingencia IA.	Cumplido
OE3 Producto funcional	Capítulo 4 muestra pantallas reales, API, MongoDB, chatbot LM Studio y panel administrador.	Cumplido

Objetivo	Evidencia de cumplimiento	Valoración
OE4 Evaluación	Pruebas automáticas, npm run verify, auditoría diseño-código y revisión de reglas críticas.	Cumplido

5.3 CUMPLIMIENTO DE OBJETIVOS TRANSVERSALES

Privacidad y seguridad. El modelo se ejecuta localmente mediante LM Studio, por lo que la conversación no necesita enviarse a un proveedor externo. El panel se protege con contraseñas almacenadas como hash bcrypt, JWT, middleware de autorización, cabeceras Helmet, limitación de peticiones y cierre de sesión ante respuestas 401.

Usabilidad y coherencia operativa. La web ofrece un acceso directo al chatbot y un catálogo numerado para reducir ambigüedades. Los mensajes de validación indican cómo corregir nombre, servicio, fecha u hora, mientras que el panel reúne estadísticas, filtros y acciones de agenda en recorridos breves.

Mantenibilidad y calidad. Frontend, API, servicios, modelos e integración con LM Studio mantienen responsabilidades separadas. El catálogo se centraliza, las reglas de citas se reutilizan en chat y administración, la trazabilidad RUP enlaza UC con código y pruebas, y npm run verify comprueba sintaxis, pruebas automáticas y construcción de producción.

5.4 EVALUACIÓN DE EFICIENCIA E INTEGRIDAD

La eficiencia del sistema se evalúa desde dos perspectivas. La primera es técnica: el backend debe validar y persistir citas sin inconsistencias. La segunda es operativa: el usuario debe poder completar una reserva sin navegar por formularios largos. El tiempo exacto de generación del LLM depende del equipo local y del modelo cargado en LM Studio, por lo que la aplicación no fija una cifra universal; en su lugar incorpora timeout configurable, endpoint de salud y respuesta de error controlada.

Tabla 26

5.4 Evaluación de eficiencia e integridad

Aspecto evaluado	Mecanismo	Resultado
Integridad de citas	Validación de nombre, servicio, horario, fecha futura y solape.	No se persiste una reserva inválida.
Coherencia de catálogo	Sincronización de servicios desde SERVICE_CATALOG.	Los servicios 1..7 son consistentes en prompt, API y MongoDB.
Disponibilidad de IA	Timeout LMSTUDIO_TIMEOUT_MS y endpoint /api/health/lmstudio.	Fallo controlado si LM Studio no responde.
Rendimiento frontend	Build Vite de producción.	Aplicación preparada para servir assets optimizados.
Regresión funcional	node:test sobre servicios críticos.	44 pruebas automáticas superadas.
Operación administrativa	Filtros, orden y estados en panel.	El peluquero puede consultar y cerrar el ciclo de vida de la cita.

5.5 DISCUSIÓN DE RESULTADOS

La decisión más relevante del proyecto ha sido separar conversación y decisión. LM Studio aporta naturalidad, tono y flexibilidad lingüística; sin embargo, el sistema no le concede autoridad final sobre la reserva. Esta arquitectura reduce el riesgo de alucinaciones operativas: aunque el modelo produzca texto incorrecto, la cita solo se registra si supera las reglas del backend.

Otra decisión importante ha sido utilizar MongoDB. En una agenda de citas, cada reserva se consulta como unidad documental completa y no requiere un modelo relacional complejo. El esquema usado conserva los datos necesarios para el panel, las estadísticas y la

detección de solapes. Además, Mongoose aporta validación de esquema, índices y una capa de acceso coherente con Node.js.

Desde el punto de vista metodológico, la inspiración en pySigHor ha sido especialmente útil para no dejar el TFG como una simple implementación. La carpeta RUP actúa como memoria viva del proyecto: casos de uso, código, pruebas y auditoría pueden recorrerse juntos. Esta trazabilidad facilita justificar decisiones ante el tutor y ante un tribunal, porque cada pantalla se conecta con un caso de uso y cada regla crítica con una prueba.

Tabla 27

5.5 Discusión de resultados

Decisión	Ventaja	Compromiso asumido
IA local con LM Studio	Privacidad y ausencia de coste por llamada.	Dependencia de que el equipo local tenga el modelo cargado.
Reglas deterministas en backend	Mayor seguridad funcional frente a respuestas impredecibles.	Más lógica propia que mantener.
React/Vite	Interfaz rápida de desarrollar, modular y moderna.	Necesidad de build separado para producción.
Node.js/Express	API ligera, comprensible y suficiente para MVC.	Menos estructura impuesta que frameworks más opinados.
MongoDB/Mongoose	Modelo documental simple y flexible.	Requiere cuidar índices y validaciones para mantener consistencia.

5.6 LIMITACIONES DETECTADAS

Las limitaciones no invalidan la solución; delimitan el alcance realista de una primera versión de TFG. La más evidente es que el chatbot depende del servidor local de LM Studio.

Si el modelo no está cargado o el puerto no responde, el sistema informa del problema y evita confirmar reservas inventadas, pero la experiencia conversacional queda temporalmente indisponible.

- La disponibilidad del asistente depende de que LM Studio esté abierto, el modelo cargado y el endpoint local activo.
- La agenda no incorpora aún notificaciones automáticas por correo, SMS o WhatsApp.
- El sistema está parametrizado para una única peluquería y no para una red de establecimientos.
- La autenticación cubre al administrador, pero no existe todavía un área privada para clientes finales.
- La evaluación se centra en pruebas funcionales y de reglas de negocio; no se ha realizado un estudio formal con usuarios reales.

5.7 RECOMENDACIONES

Para una puesta en uso real conviene mantener el mismo criterio que ha guiado el desarrollo: avanzar por iteraciones pequeñas, con trazabilidad y pruebas. No sería recomendable añadir funcionalidades de forma masiva sin consolidar primero la operación diaria del peluquero.

- Mantener el backend como autoridad de negocio: ningún cambio del prompt debe sustituir validaciones de agenda.
- Configurar variables de entorno propias antes de un despliegue real, especialmente JWT_SECRET, usuario administrador y URI de MongoDB.

- Añadir copias de seguridad periódicas de MongoDB si la aplicación se usa con clientes reales.
- Registrar conversaciones solo si existe consentimiento y una política clara de conservación de datos.
- Ampliar las pruebas automáticas cada vez que se añada una regla de negocio o un nuevo estado de cita.

5.8 FUTURAS LÍNEAS DE ACTUACIÓN

La solución queda preparada para evolucionar. La estructura MVC, el catálogo centralizado y la separación entre IA, negocio y persistencia permiten añadir funcionalidades sin reescribir el núcleo.

Tabla 28

5.8 Futuras líneas de actuación

Línea futura	Descripción	Prioridad
Disponibilidad avanzada	Mostrar huecos libres calculados automáticamente y sugerir alternativas concretas.	Alta
Recordatorios	Enviar avisos por email, SMS o WhatsApp antes de la cita.	Alta
Cancelación por cliente	Permitir cancelar o cambiar una reserva mediante enlace seguro o conversación.	Media
Calendario visual	Añadir vista semanal/mensual para el peluquero.	Media
Multiusuario	Soportar varios peluqueros, turnos y servicios por empleado.	Media
Analítica	Informes de servicios más solicitados, horas punta e ingresos por periodo.	Media

Línea futura	Descripción	Prioridad
Mejora del chatbot	Añadir pruebas conversacionales de regresión y evaluación de calidad de respuesta.	Alta
Despliegue controlado	Preparar ejecución en red local o servidor privado manteniendo IA local.	Baja-media

5.9 VALORACIÓN PERSONAL DEL PROCESO

El proyecto ha permitido recorrer un ciclo completo de ingeniería de software: entender una necesidad real, modelarla, diseñar una arquitectura, implementar una solución y verificar sus reglas críticas. La parte más valiosa no ha sido únicamente integrar un LLM, sino aprender a integrarlo con responsabilidad: la IA mejora la experiencia de usuario, pero no reemplaza el diseño de dominio ni la validación de negocio.

También se confirma la utilidad de documentar el proceso. Los capítulos, los diagramas, la carpeta RUP y las pruebas no son elementos aislados; forman una cadena de evidencia. Esa cadena es la que permite defender que la aplicación no se ha construido de forma improvisada, sino siguiendo un razonamiento técnico progresivo.

5.10 CONCLUSIÓN FINAL

Corte Perfecto demuestra que una pequeña empresa puede beneficiarse de tecnologías actuales sin asumir una arquitectura desproporcionada ni renunciar a la privacidad. La combinación de React, Node.js, MongoDB y LM Studio permite construir una solución funcional, local, mantenible y alineada con el RGPD. El resultado final satisface los objetivos planteados y deja una base técnica suficiente para seguir evolucionando el sistema en iteraciones posteriores.

REFERENCIAS

- Express.js. (s. f.). *Express—Node.js web application framework*. <https://expressjs.com/>
- Følstad, A., & Brandtzæg, P. B. (2017). Chatbots and the new world of HCI. *Interactions*, 24(5), 38–42. <https://doi.org/10.1145/3085558>
- LM Studio. (s. f.). *LM Studio docs*. <https://lmstudio.ai/docs>
- Meta AI. (2024, julio 23). *The Llama 3 herd of models*.
<https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>
- MongoDB, Inc. (s. f.). *MongoDB manual*. <https://www.mongodb.com/docs/manual/>
- Mongoose. (s. f.). *Mongoose documentation*. <https://mongoosejs.com/docs/>
- OpenJS Foundation. (s. f.). *Node.js API documentation*. <https://nodejs.org/docs/latest/api/>
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
- React. (s. f.). *React documentation*. <https://react.dev/>
- Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos. Diario Oficial de la Unión Europea, L 119, 1–88. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., ... Scialom, T. (2023). *Llama 2: Open foundation and fine-tuned chat models* [Preprint]. arXiv.
<https://doi.org/10.48550/arXiv.2307.09288>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., &

Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information*

Processing Systems, 30, 5998–6008. <https://proceedings.neurips.cc/paper/7181->

[attention-is-all-you-need](https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need)

Vite. (s. f.). *Vite guide*. <https://vite.dev/guide/>